

Approximate Matching

Ben Langmead



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

Department of Computer Science



Please sign guestbook (www.langmead-lab.org/teaching-materials) to tell me briefly how you are using the slides. For original Keynote files, email me (ben.langmead@gmail.com).

Read

CTCAAACCTCCTGACCTTTGGTGATCCACCCGCCTNGGCCTTC

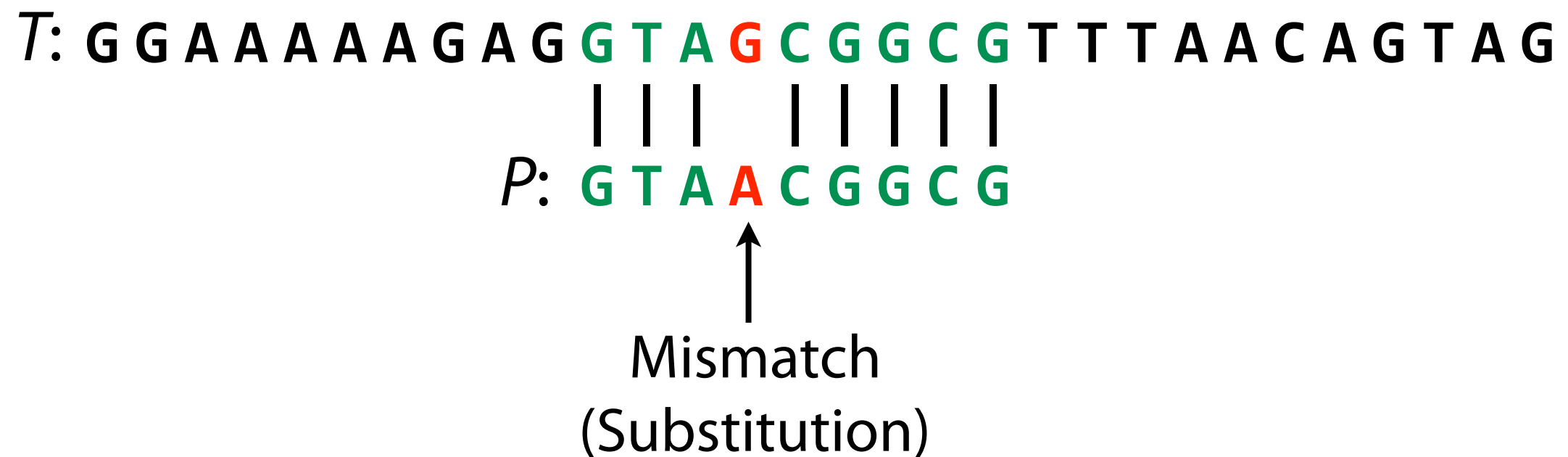
Reference

GATCACAGGTCTATCACCCCTATTAACCACTCACGGGAGCTCTCCATGCATTTGGTATTTT
CGTCTGGGGGGTATGCACGCGATAGCATTGCGAGACGCTGGAGCCGGAGCACCCCTATGTC
GCAGTATCTGTCTTTGATTCTGCCTCATCTATTATTTATCGCACCTACGTTCAATATT
ACAGGCGAACATACTTACTAAAGTGTGTTAATTAATTAATGCTTGTAGGACATAATAATA
ACAATTGAATGTCTGCACAGCCACTTTCCACACAGACATCATAACAAAAAATTTCCACCA
AACCCCCCTCCCCGCTTCTGGCCACAGCACTTAAACAGCTCTGCCAAACCCCAAAA
ACAAAGAACCCTAACACCAGCCTAACCAATTTCAAATTTTATCTTTGGCGGTATGCAC
TTTTAACAGTCACCCCCCACTAACCAATTTTCCCCTCCCCTCTCACTACTACTAAT
CTCATCAATAACAACCCCCGCCATCTACCCAGCACACACACACCCCTCTAACCCCAT
CCCCGAACCAACCAACCCCAAGCCACCCCCACAGTTTATGTAGCTCTCTCTCTCAAA
GCAATACACTGACCCGCTCAAACCTCTGGATTTTGGATCCACCCAGCGCTTTGGCCTAAA
CTAGCCTTTCTATTAGCTCTTAGAAGATTACACATGCAAGCATCCCCGCTCCAGTGAGT
TCACCCTCTAAATCACCAGCATCAAGGAACAAGCATCAAGCACGCAGCAATGCAGCTC
AAAACGCTTAGCCTAGCCACACCCCTCACGGGAAACAGCAGTGATTAACCTTAGCAATAA
ACGAAAGTTTAACTAAGCTATACTTACCCAGGGTTGGTCAATTTCTGCTCCAGCCACCGC
GGTCACACGATTAACCCAAGTCAATGAAGCCGGCGTAAAGAGTGTCTAGATCACCCCC
TCCCCAATAAAGCTAAAACTCACCTGATTTGTAAAAAACTCCAGTTTACAAAATAGAC
TACGAAAGTGGCTTTAACATATCTGAACAACAATAGCTAAGCACTGGGATTAGA
TACCCCACTATGCTTAGCCCTAAACCTCAACAGTCAACAACCTGAGCCAGAA
CACTACGAGCCACAGCTTAAAACTCAAAGGACCTGGCGGTGCTTCATCTAGAGG
AGCCTGTTCTGTAATCGATAAACCCCGATCAACCTCACCACCTCTTGCTTATA
CCGCCATCTTCAGCAAACCCTGATGAAGGCTACAAAGTAAGCGCAAGTACCTAG
ACGTTAGGTCAAGGTGTAGCCCATGAGGTGGCAAGAAATGGGCTACATTTTC
AAAACTACGATAGCCCTTATGAACTTAAAGGTGCAAGGTGGATTTAGCAGTAA
AGTAGAGTGCTTAGTTGAACAGGGCCCTGAAGCGGTACACACCCGCCGTCACCC
AAGTATACTTCAAAGGACATTTAACTAAACCCCTACGCATTTATATAGAGGAGACAA
CGTAACCTCAAACCTCCTGCCTTTGGTGATCCACCCGCCTTGGCCTACCTGCATAATGAAG
AAGCACCCAACTTACACTTAGGAGATTTCACTTAACTTGACCGCTCTGAGCTAAACCTA
GCCCCAAACCCACTCCACCTTACTACCAGACAACCTTAGCCAAACCATTTACCCAAATAA
AGTATAGGCGATAGAAATTGAAACCTGGCGCAATAGATATAGTACCGCAAGGGAAAGATG
AAAAATTATAACCAAGCATAATATAGCAAGGACTAACCCCTATACCTTCTGCATAATGAA
TTAACTAGAAATAACTTTGCAAGGAGAGCCAAAGCTAAGACCCCGAAACCAGACGAGCT
ACCTAAGAACAGCTAAAGAGCACACCCGCTCTATGTAGCAAAATAGTGGGAAGATTTATA
GGTAGAGGCGACAAACCTACCGAGCCTGGTGATAGCTGGTTGTCCAAGATAGAATCTTAG
TTCAACTTTAAATTTGCCACAGAACCTCTAAATCCCCTTGTAATTTAACTGTTAGTC
CAAAGAGGAACAGCTCTTTGGACACTAGGAAAAAACCTTGTAAGAGAGAGTAAAAAATTA
ACACCCATAGTAGGCCTAAAAGCAGCCACCAATTAAGAAAGCGTTCAAGCTCAACACCCA
CTACCTAAAAAATCCCAAACATATAACTGAACTCCTCACACCCAATTGGACCAATCTATC
ACCCTATAGAAGAACTAATGTTAGTATAAGTAACATGAAAACATTCTCCTCCGCATAAGC
CTGCGTCAGATTAACCACTGAACTGACAATTAACAGCCCAATATCTACAATCAACCAAC
AAGTCATTATTACCCTCACTGTCAACCCAACACAGGCATGCTCATAAGGAAAGGTTAAAA
AAAGTAAAAGGAACTCGGCAAATCTTACCCCGCCTGTTTACCAAAAACATCACCTCTAGC
ATCACCAGTATTAGAGGCACCGCCTGCCAGTGACACATGTTTAAACGGCCGCGGTACCCT
AACCGTGCAAAGCTAGCATAATCACTTGTTCCTTAAATAGCGACCTGTATGAATCGCTCC

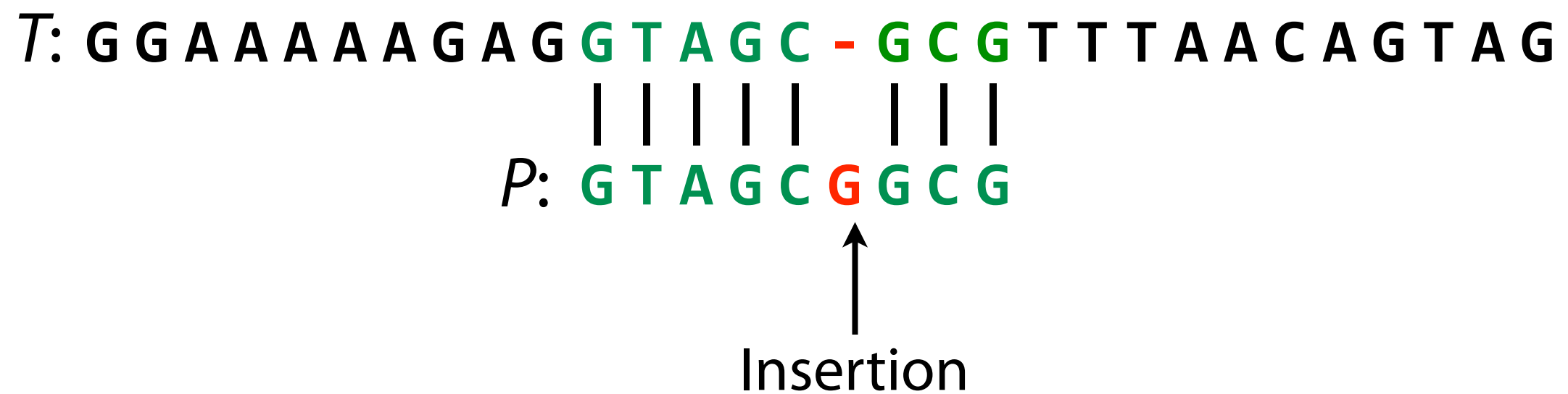
Sequence differences occur because of...

1. Sequencing error
2. Genetic variation

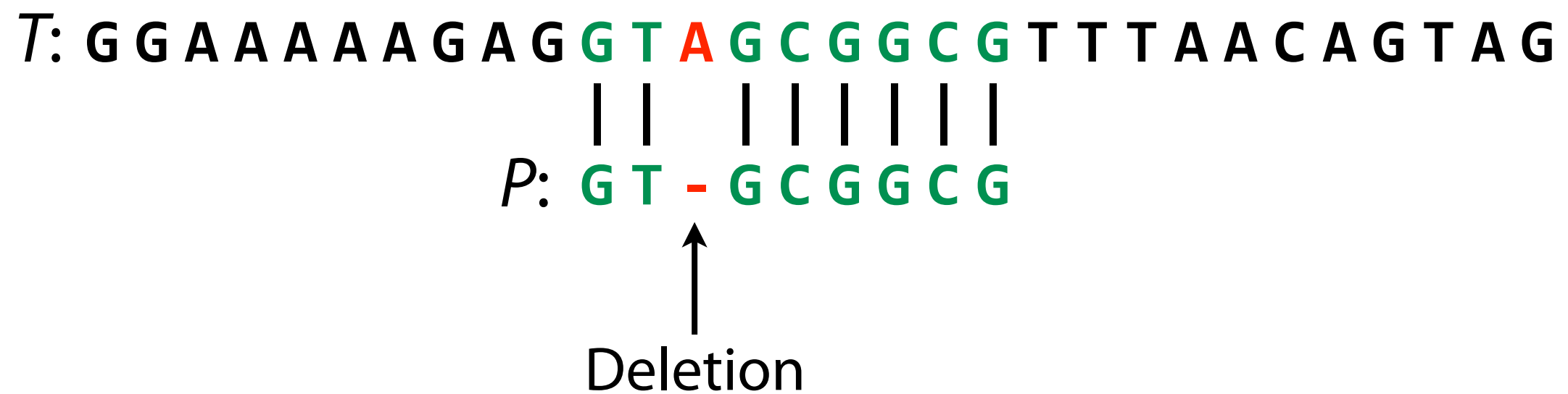
Approximate matching



Approximate matching



Approximate matching



Hamming distance

For X & Y where $|X| = |Y|$, *hamming distance* =
minimum # substitutions needed to turn one into the other

$X:$	G	A	G	G	T	A	G	C	G	G	C	G	T	T
$Y:$	G	T	G	G	T	A	A	C	G	G	G	G	T	T

Hamming distance = 3

Edit distance

(AKA Levenshtein distance)

For X & Y , *edit distance* = minimum # edits (substitutions, insertions, deletions) needed to turn one into the other

X: T G G C C G C G C A A A A A C A G C
| | | | | | | | | | | | | | |

Y: T G A C C G C G C A A A A - C A G C

Edit distance = 2

X: G C G T A T G C G G C T A - A C G C
| | | | | | | | | | | | |

Y: G C - T A T G C G G C T A T A C G C

Edit distance = 2

Approximate matching

Like exact matching, but *pattern* P may be within a certain *distance* (usually Hamming or edit) of T . Each such place is an *approximate match*.

Allowing edits is more challenging than just allowing mismatches

We'll return to edits

Approximate matching

```
def naive(p, t):
    occurrences = []
    for i in range(len(t) - len(p) + 1):  # Loop over alignments
        match = True
        for j in range(len(p)):           # Loop over characters
            if t[i+j] != p[j]:             # compare characters
                match = False              # mismatch; reject alignment
                break
        if match:
            occurrences.append(i)          # all chars matched; record
    return occurrences
```

Approximate matching

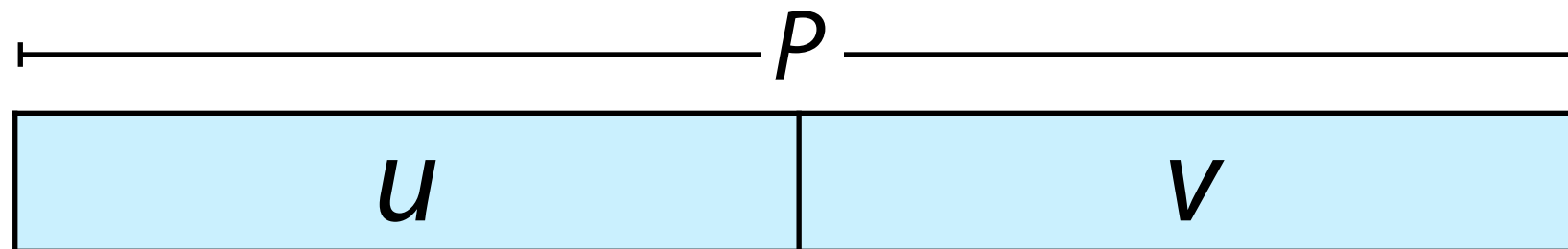
```
def naive_approx_hamming(p, t, maxDistance):
    occurrences = []
    for i in range(len(t) - len(p) + 1):  # Loop over alignments
        nmm = 0
        for j in range(len(p)):           # Loop over characters
            if t[i+j] != p[j]:             # compare characters
                nmm += 1                   # mismatch
            if nmm > maxDistance:
                break                       # exceeded max hamming dist
        if nmm <= maxDistance:
            occurrences.append(i)          # approximate match
    return occurrences
```

http://bit.ly/CG_NaiveApprox

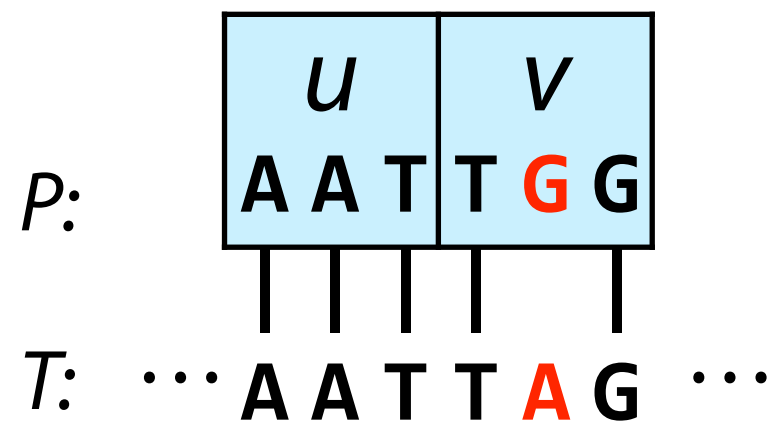
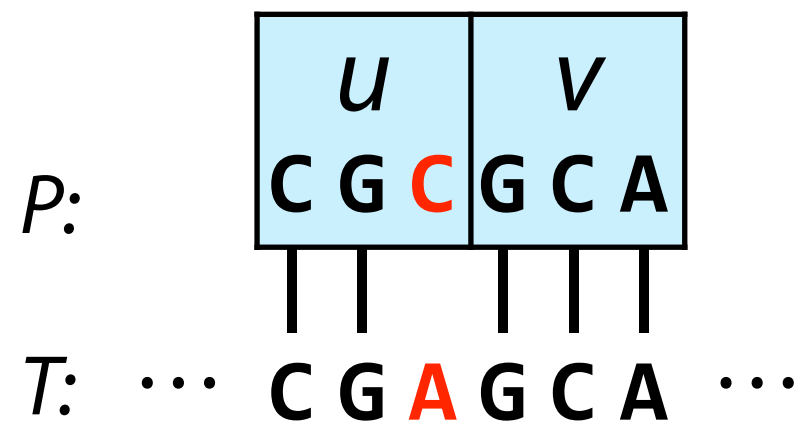
Approximate matching

Wanted: way to apply exact matching algorithms to approximate matching problems

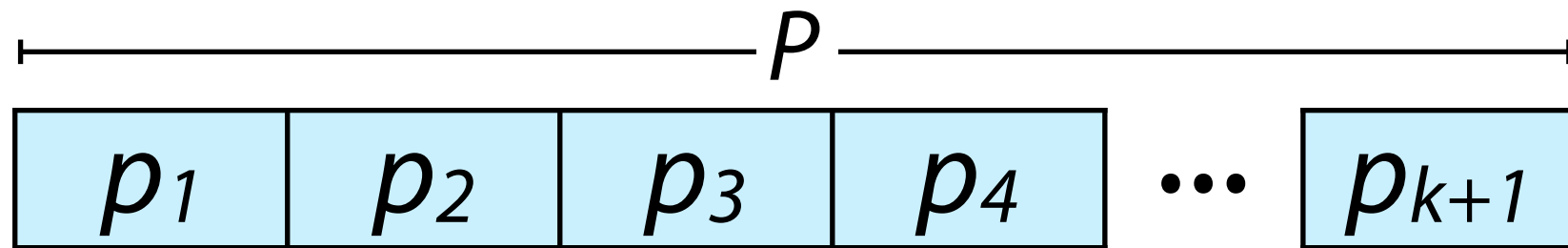
Approximate matching



If P occurs in T with 1 edit, then u or v appears with no edits

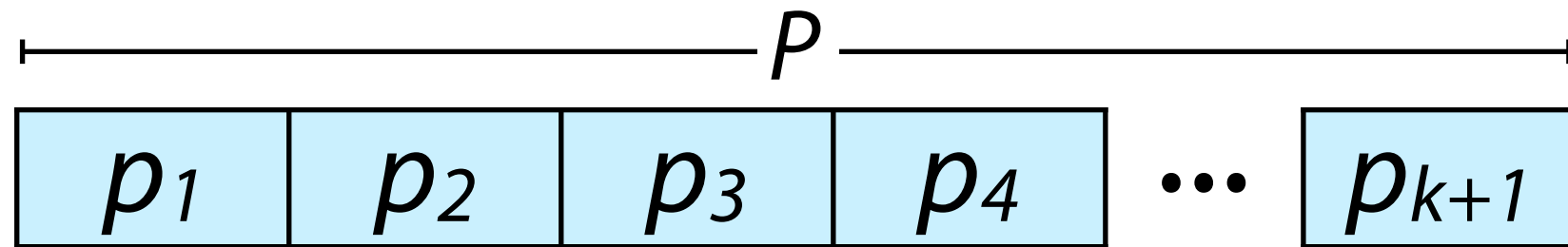


Approximate matching



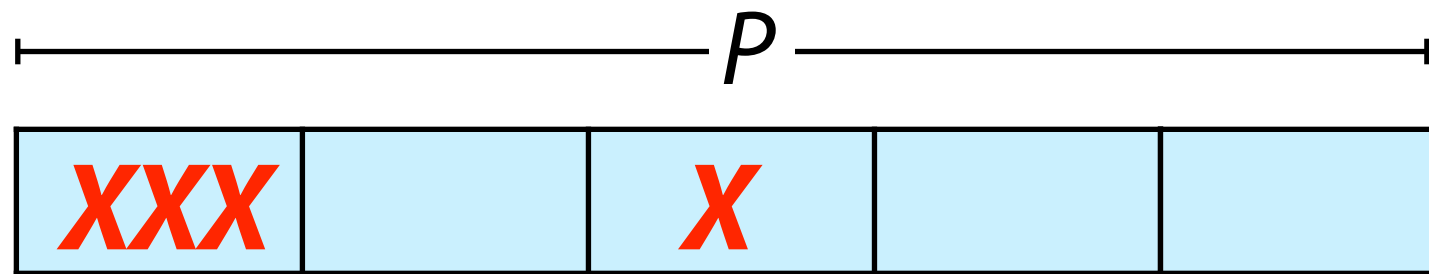
If P occurs in T with up to k edits...

Approximate matching

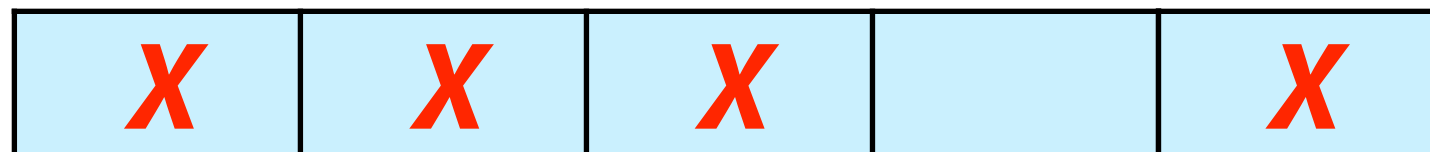
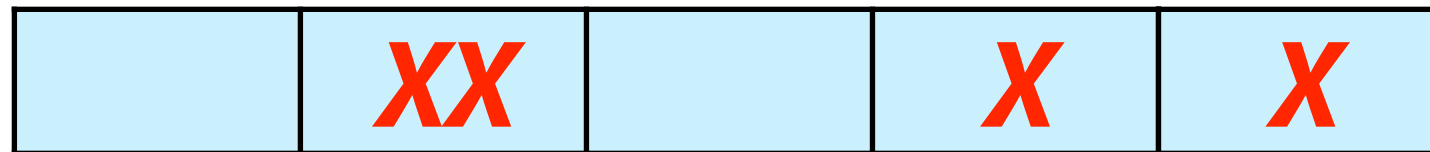


If P occurs in T with up to k edits, then at least one of $p_1, p_2, \dots, p_{k+1}$ must appear with 0 edits

Approximate matching



5 partitions
4 edits (**X**)



Approximate matching



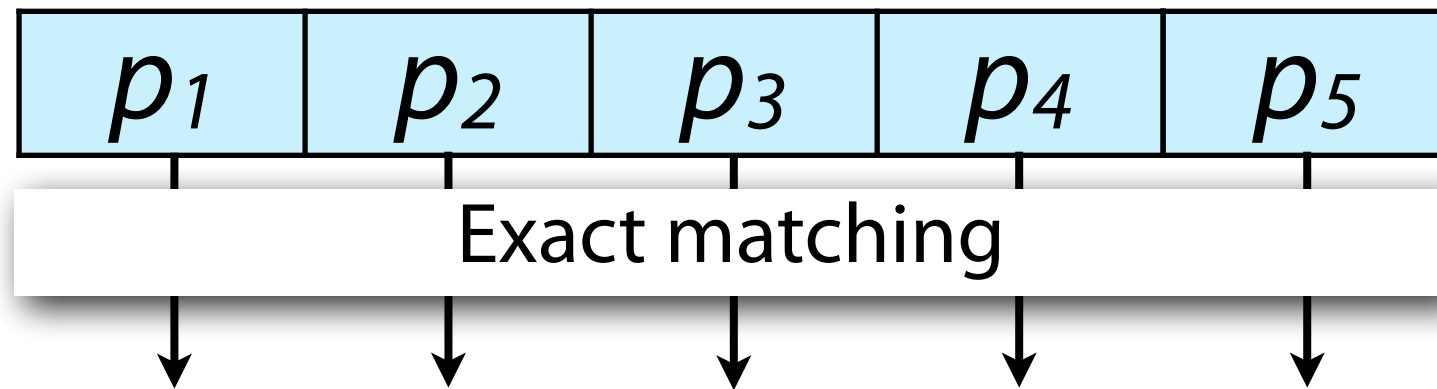
Pigeonhole principle: $k+1$ pigeons, k holes.
At least one has >1 pigeon!

Approximate matching



We have k pigeons, $k+1$ holes, at least one...
...is *empty*

Pigeonhole principle



T

What algorithm can we use?

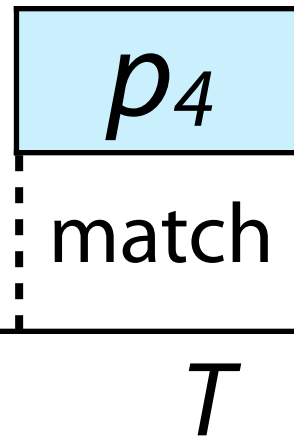
Any exact matching algorithm

If we have a k-mer index, we can use that

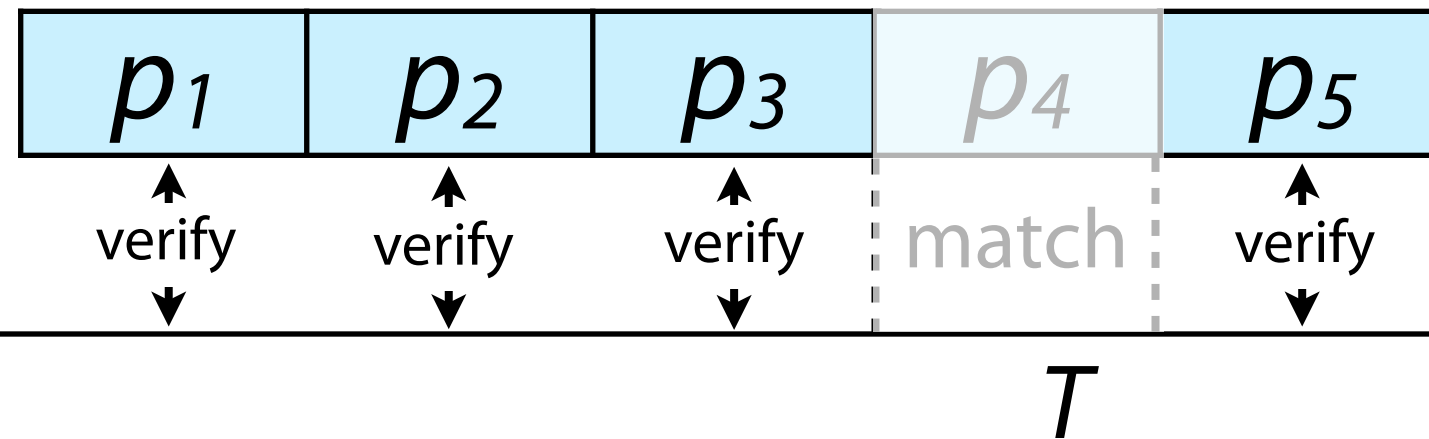
Naive exact matching

Boyer-Moore

Pigeonhole principle

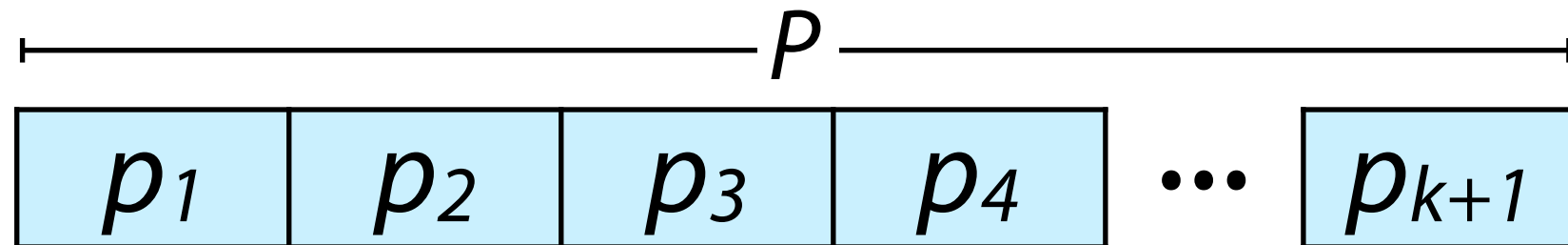


Pigeonhole principle



For Hamming distance, verification is essentially just the inner loop of **naive_approx_hamming** from before

Pigeonhole principle



Advantages

Reuse favorite exact matching algos; fast and easy

Flexible; works for Hamming and edit distance*

Disadvantages

Large k yields small partitions matching many times by chance; lots of verification work

$k+1$ exact matching problems, one per partition

* we don't know how to do edit distance verification yet

Dynamic Programming & Edit Distance

Ben Langmead



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

Department of Computer Science



Please sign guestbook (www.langmead-lab.org/teaching-materials) to tell me briefly how you are using the slides. For original Keynote files, email me (ben.langmead@gmail.com).

Approximate matching for biosequences

Some widely used approximate matching algorithms for DNA (& other strings) came from the biological community, aiming to bring alignments into closer harmony with biological processes that mutate strings

Score = 248 bits (129), Expect = 1e-63
Identities = 213/263 (80%), Gaps = 34/263 (12%)
Strand = Plus / Plus

```
Query: 161 atatcaccacgtcaaagggtgactccaactcca---ccactccattttgttcagataaatgc 217
      ||||||||||||||||||||||||||||| | | | ||| |||||||||||||||
Sbjct: 481 atatcaccacgtcaaagggtgactccaact-tattgatagtgttttatgttcagataaatgc 539
```

```
Query: 218 ccgatgatcatgtcatgcagctccaccgattgtgagaacgacagcgacttccgtcccagc 277
        |||||  |||||||||||||||||  ||  |||||
Sbjct: 540 ccgatgactttgtcatgcagctccaccgattttg-g-----ttccgtcccagc 586
```

```
Query: 278 c-gtgcc--aggtgctgcctcagattcaggttatgccgctcaattcgctgcgtatatcgc 334
        | ||| | ||||||||| ||||||||| ||||||||| ||||||||| |||||||||
Sbjct: 587 caatgacgta-gtgctgcctcagattcaggttatgccgctcaattcgctgggtatatcgc 645
```

```
Query: 335 ttgctgattacgtgcagctttcccttcaggcggga-----ccagccatccgtc 382
      |||||||||||||||||||||||||||||||||
Sbjct: 646 ttgctgattacgtgcagctttcccttcaggcgggattcatacagcggccagccatccgtc 705
```

```

Query: 383 ctccatatc-accacgtcaaagg 404
      ||||| |||||
Sbjct: 706 atccatatcaaccacgtcaaagg 728

```


Hamming / edit distance

For X, Y where $|X| = |Y|$, *hamming distance* = minimum # substitutions needed to turn one into the other

For X, Y , *edit distance* = minimum # edits (substitutions, insertions, deletions) needed to turn one into the other

Finding distances

```
def hammingDistance(x, y):
```

```
    ????
```

Strategy: walk along both strings. For each position, compare the characters in both strings at that position. If not equal, increment hamming distance:

x:	G	C	G	T	A	T	G	C	G	G	C	T	A	A	C	G	C	
y:	G	C	T	T	A	T	G	C	G	G	C	T	A	T	A	C	G	C

Finding distances

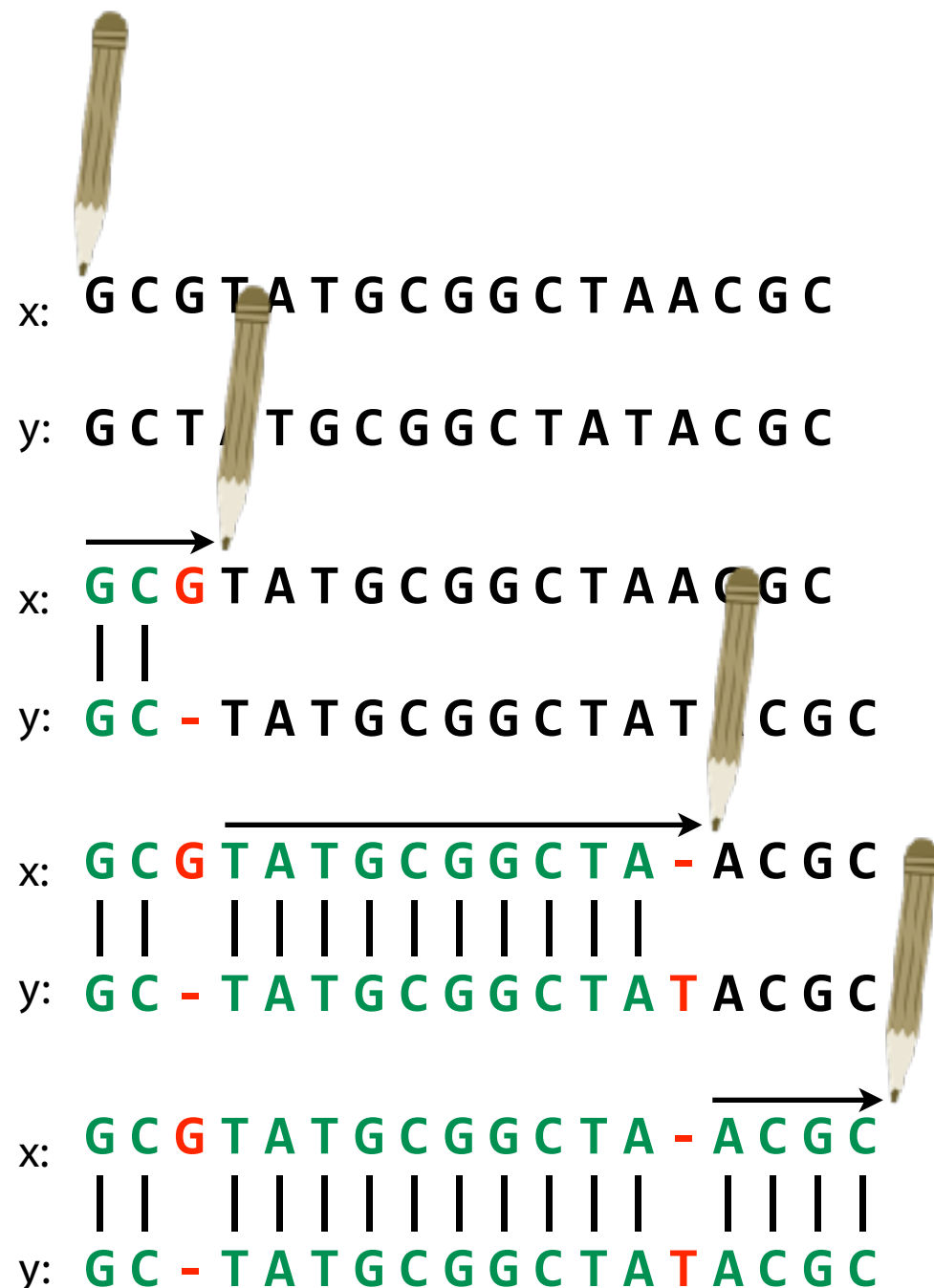
```
def hammingDistance(x, y):  
    nmm = 0  
    for i in range(0, len(x)):  
        if x[i] != y[i]:  
            nmm += 1  
    return nmm
```

```
def editDistance(x, y):  
    ????
```

Finding Hamming distance between strings is pretty easy.
What about edit distance?

Edit distance

Imagine edits are introduced by an *optimal editor* working left-to-right:



Edit transcript summarizes how editor turns *x* into *y*

Operations:

M = match, **R** = replace,

I = insert into *x*, **D** = delete from *x*

MMD

Reminder: this is **D**, not **I**, because we have to delete a character from *x* to make it more like *y*

MMDMMMMMMMMMMI

MMDMMMMMMMMMMIMMMM

Edit distance

Alignments:

x: G C G T A T G C G G C T A - A C G C
| | | | | | | | | | | | | |
y: G C - T A T G C G G C T A T A C G C

replacement (AKA substitution/mismatch)

x: G C G T A T G A G G C T A - A C G C
| | | | | | | | | | | | | |
y: G C - T A T G C G G C T A T A C G C

x: t h e l o n g e s t - - - -
| | | | | | | |
y: - - - - l o n g e s t d a y

Edit transcripts (w/r/t x):

M M D M M M M M M M M M M I M M M M

Distance = 2

M M D M M M M R M M M M M I M M M M

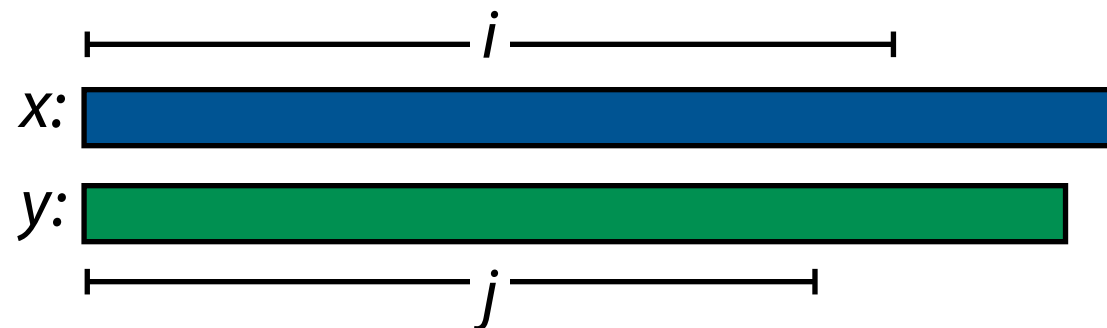
Distance = 3

D D D D M M M M M M M I I I I

Distance = 8

Edit distance

$D[i, j]$: edit distance between length- i prefix of x and length- j prefix of y



Optimal edit transcript for $D[i, j]$ is built by extending a shorter one by 1 operation. 3 options:

Append **D** to transcript for $D[i-1, j]$

Append **I** to transcript for $D[i, j-1]$

Append **M** or **R** to transcript for $D[i-1, j-1]$

$D[i, j]$ and its transcript come from 1 of these 3 choices, whichever has fewest edits

$D[|x|, |y|]$ is the overall edit distance

Edit distance

Is at beginning



Ds at beginning



Let $D[0, j] = j$, and let $D[i, 0] = i$

Otherwise, let $D[i, j] = \min \begin{cases} D[i-1, j] + 1 \\ D[i, j-1] + 1 \\ D[i-1, j-1] + \delta(x[i-1], y[j-1]) \end{cases}$

vertical (**D**)

horizontal (**I**)

diagonal (**M** or **R**)

$\delta(a, b)$ is 0 if $a = b$, 1 otherwise

Edit distance: dynamic programming

ε is empty string

ε G C T A T G C C A C G C

ε

G

C

G

T

A

T

G

C

A

C

G

C

D: x

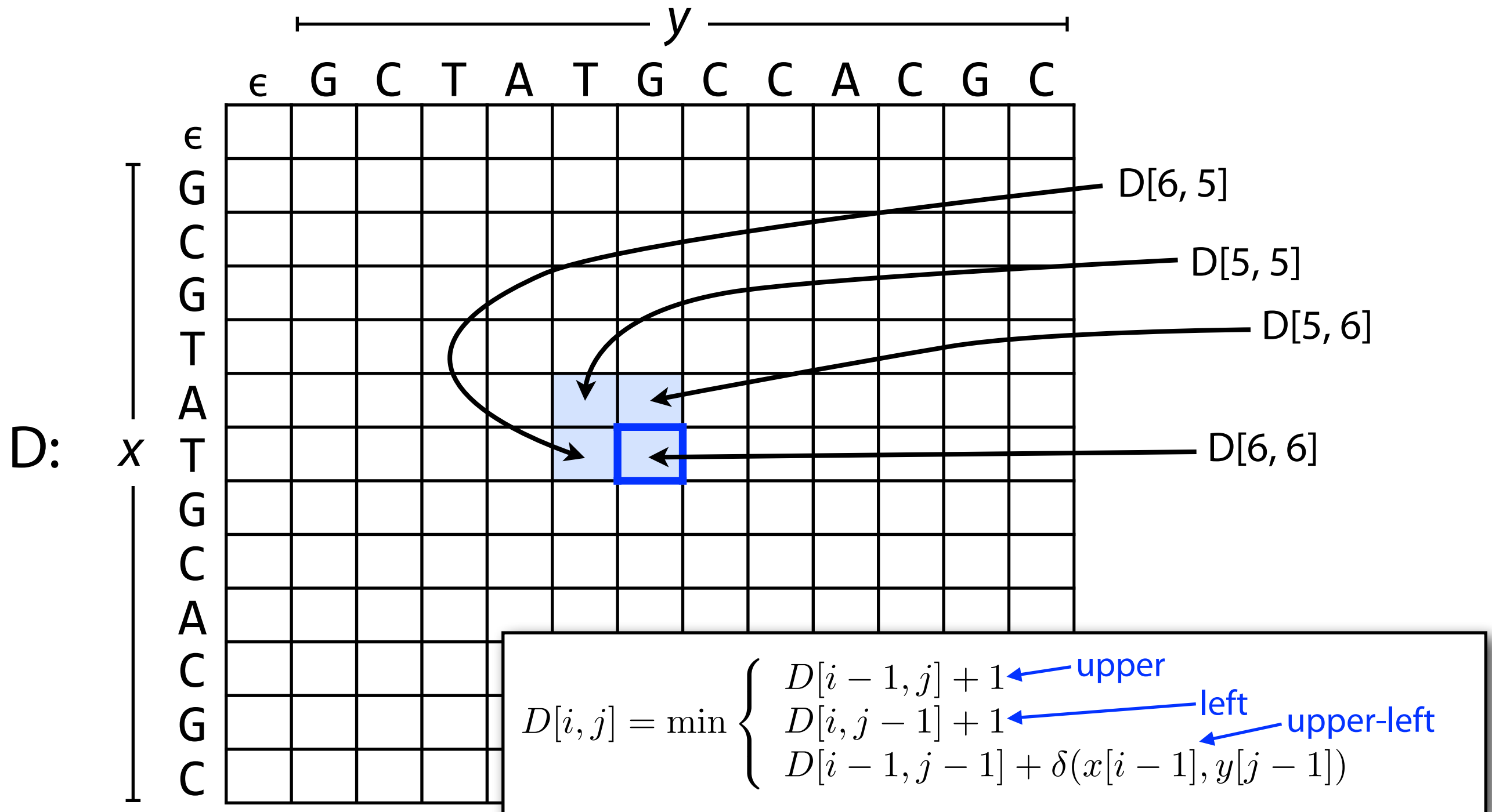
ε													
G													
C													
G													
T													
A													
T													
G													
C													
A													
C													
G													
C													

Let $n = |x|$, $m = |y|$

D: $(n+1) \times (m+1)$ matrix

$D[i, j]$ = edit distance b/t
length- i prefix of x and
length- j prefix of y

Edit distance: dynamic programming



Cell depends upon its upper, left, and upper-left neighbors

Edit distance: dynamic programming

First few lines
of `edDistDp`:

```
D = numpy.zeros((len(x)+1, len(y)+1), dtype=int)
D[0, 1:] = range(1, len(y)+1)
D[1:, 0] = range(1, len(x)+1)
```

	ε	G	C	T	A	T	G	C	C	A	C	G	C
ε	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1												
C	2												
G	3												
T	4												
A	5												
T	6												
G	7												
C	8												
A	9												
C	10												
G	11												
C	12												

Initialize $D[0, j]$ to j ,
 $D[i, 0]$ to i

Edit distance: dynamic programming

Loop from
edDistDp:

```

for i in range(1, len(x)+1):
    for j in range(1, len(y)+1):
        delt = 1 if x[i-1] != y[j-1] else 0
        D[i, j] = min(D[i-1, j-1]+delt, D[i-1, j]+1, D[i, j-1]+1)
    
```

	ε	G	C	T	A	T	G	C	C	A	C	G	C
ε	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1												
C	2												
G	3												
T	4												
A	5												
T	6												
G	7												
C	8												
A	9												
C	10												
G	11												
C	12												

Fill remaining cells from
top row to bottom and
from left to right

Edit distance: dynamic programming

Loop from
edDistDp:

```

for i in range(1, len(x)+1):
    for j in range(1, len(y)+1):
        delt = 1 if x[i-1] != y[j-1] else 0
        D[i, j] = min(D[i-1, j-1]+delt, D[i-1, j]+1, D[i, j-1]+1)
    
```

	ε	G	C	T	A	T	G	C	C	A	C	G	C
ε	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	?											
C	2												
G	3												
T	4												
A	5												
T	6												
G	7												
C	8												
A	9												
C	10												
G	11												
C	12												

Fill remaining cells from
top row to bottom and
from left to right

What goes here in $i=1, j=1$?

$x[i-1] = y[j-1] = 'G'$,

SO $delt = 0$

```

D[i, j] = min(D[i-1, j-1]+delt,
              D[i-1, j]+1,
              D[i, j-1]+1)
          = min(0 + 0, 1 + 1, 1 + 1)
          = 0
    
```

Edit distance: dynamic programming

```
Loop from      for i in range(1, len(x)+1):
edDistDp:      for j in range(1, len(y)+1):
                delt = 1 if x[i-1] != y[j-1] else 0
                D[i, j] = min(D[i-1, j-1]+delt, D[i-1, j]+1, D[i, j-1]+1)
```

	ε	G	C	T	A	T	G	C	C	A	C	G	C
ε	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2

Fill remaining cells from
top row to bottom and
from left to right

← Edit distance for x, y

Edit distance: dynamic programming

	ε	G	C	T	A	T	G	C	C	A	C	G	C
ε	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2

← Edit distance for x, y

But where and what are
the 2 edits?

Edit distance: getting the alignment

Traceback corresponds to an optimal alignment / edit transcript

At each step, ask: which neighbor ($\nwarrow$, $\leftarrow$ or $\nearrow$) gave the minimum?

	ε	G	C	T	A	T	G	C	C	A	C	G	C
ε	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2

A: From here

Q: How did I get here?

Edit distance: getting the alignment

Traceback corresponds to an optimal alignment / edit transcript

At each step, ask: which neighbor ($\nwarrow$, $\leftarrow$ or $\nearrow$) gave the minimum?

	ε	G	C	T	A	T	G	C	C	A	C	G	C
ε	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2

A: From here

Q: How did I get here?

Edit distance: getting the alignment

Traceback corresponds to an optimal alignment / edit transcript

At each step, ask: which neighbor ($\nwarrow$, $\leftarrow$ or $\nearrow$) gave the minimum?

	ε	G	C	T	A	T	G	C	C	A	C	G	C
ε	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2

A: From here

Q: How did I get here?

Edit distance: getting the alignment

Traceback corresponds to an optimal alignment / edit transcript

At each step, ask: which neighbor ($\nwarrow$, $\leftarrow$ or $\nearrow$) gave the minimum?

	ε	G	C	T	A	T	G	C	C	A	C	G	C
ε	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2

A: From here

Q: How did I get here?

Edit distance: getting the alignment

Traceback corresponds to an optimal alignment / edit transcript

At each step, ask: which neighbor ($\nwarrow$, $\leftarrow$ or $\nearrow$) gave the minimum?

	ε	G	C	T	A	T	G	C	C	A	C	G	C
ε	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2

Alignment:

```
GC GT ATG - CACGC
|| |||| |||||
GC - TATG CACGC
```

Edit transcript:

```
MM D M M M M I M M M M
```

Edit distance: getting the alignment

To find the edit distance and the alignment we did:

(a) table filling, (b) traceback

If $|X| = m$ and $|Y| = n$, how much work is table filling?

Filling $(m + 1)(n + 1)$ cells, each requiring constant work, so $O(mn)$

How much work is traceback?

Each step goes $\nwarrow$, $\leftarrow$ or $\nearrow$.

Worst case: traceback never moves diagonally, requiring $m \nearrow$'s and $n \leftarrow$'s, so $O(m + n)$

	ε	G	C	T	A	T	G	C	C	A	C	G	C
ε	0	1	2	3	4	5	6	7	8	9	10	11	12
G	1	0	1	2	3	4	5	6	7	8	9	10	11
C	2	1	0	1	2	3	4	5	6	7	8	9	10
G	3	2	1	1	2	3	3	4	5	6	7	8	9
T	4	3	2	1	2	2	3	4	5	6	7	8	9
A	5	4	3	2	1	2	3	4	5	5	6	7	8
T	6	5	4	3	2	1	2	3	4	5	6	7	8
G	7	6	5	4	3	2	1	2	3	4	5	6	7
C	8	7	6	5	4	3	2	1	2	3	4	5	6
A	9	8	7	6	5	4	3	2	2	2	3	4	5
C	10	9	8	7	6	5	4	3	2	3	2	3	4
G	11	10	9	8	7	6	5	4	3	3	3	2	3
C	12	11	10	9	8	7	6	5	4	4	3	3	2

Edit distance: summary

Matrix-filling dynamic programming algorithm is $O(mn)$ time and space

Filling matrix is $O(mn)$ space and time, yields edit distance

Traceback is $O(m + n)$ time, yields optimal alignment / edit transcript

Approximate matching with edit distance

Can we search for the occurrence of P in T with least edits?

[illegible]

Approximate matching with edit distance

1. Initialize first row with 0's rather than increasing integers, then fill matrix

[illegible]

Approximate matching with edit distance

$D[i, j]$ equals the optimal edit distance between the length- i prefix of P and...

T

	ϵ	A	A	C	C	C	T	A	T	G	T	C	A	T	G	C	C	T	T	G	G	A
ϵ	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
T	1	1	1	1	1	1	0	1	0													
A	2	1	1	2	2	2	1	0	1													
C	3	2	2	1	2	2	2	1	1	2	2	1	2	2	2	1	2	2	2	2	2	2
G	4	3	3	2	2	3	3	2	2	1	2	2	2	3	2	2	2	3	3	2	2	3
T	5	4	4	3	3	3	3	3	2	2	1	2	3	2	3	3	3	2	3	3	3	3
C	6	5	5	4	3	3	4	4	3	3	2	1	2	3	3	3	3	3	3	4	4	4
A	7	6	5	5	4	4	4	4	4	4	3	2	1	2	3	4	4	4	4	4	5	4
G	8	7	6	6	5	5	5	5	5	4	4	3	2	2	2	3	4	5	5	4	4	5
C	9	8	7	6	6	5	6	6	6	5	5	4	3	3	3	2	3	4	5	5	5	5

P

T : A A C C C T A T G T C A T G C C T T G G A

P : T A C G T C A - G C

Approximate matching with edit distance

$D[i, j]$ equals the optimal edit distance between the length- i prefix of P and a substring of T ending at position j .

T

	ϵ	A	A	C	C	C	T	A	T	G	T	C	A	T	G	C	C	T	T	G	G	A
ϵ	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
T	1	1	1	1	1	1	0	1	0													
A	2	1	1	2	2	2	1	0	1													
C	3	2	2	1	2	2	2	1	1	2	2	1	2	2	2	1	2	2	2	2	2	2
G	4	3	3	2	2	3	3	2	2	1	2	2	2	3	2	2	2	3	3	2	2	3
T	5	4	4	3	3	3	3	3	2	2	1	2	3	2	3	3	3	2	3	3	3	3
C	6	5	5	4	3	3	4	4	3	3	2	1	2	3	3	3	3	3	3	4	4	4
A	7	6	5	5	4	4	4	4	4	4	3	2	1	2	3	4	4	4	4	4	5	4
G	8	7	6	6	5	5	5	5	5	4	4	3	2	2	2	3	4	5	5	4	4	5
C	9	8	7	6	6	5	6	6	6	5	5	4	3	3	3	2	3	4	5	5	5	5

T : A A C C C T A T G T C A T G C C T T G G A

P : T A C G T C A - G C

Index-Assisted Approximate Matching

Ben Langmead



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

Department of Computer Science



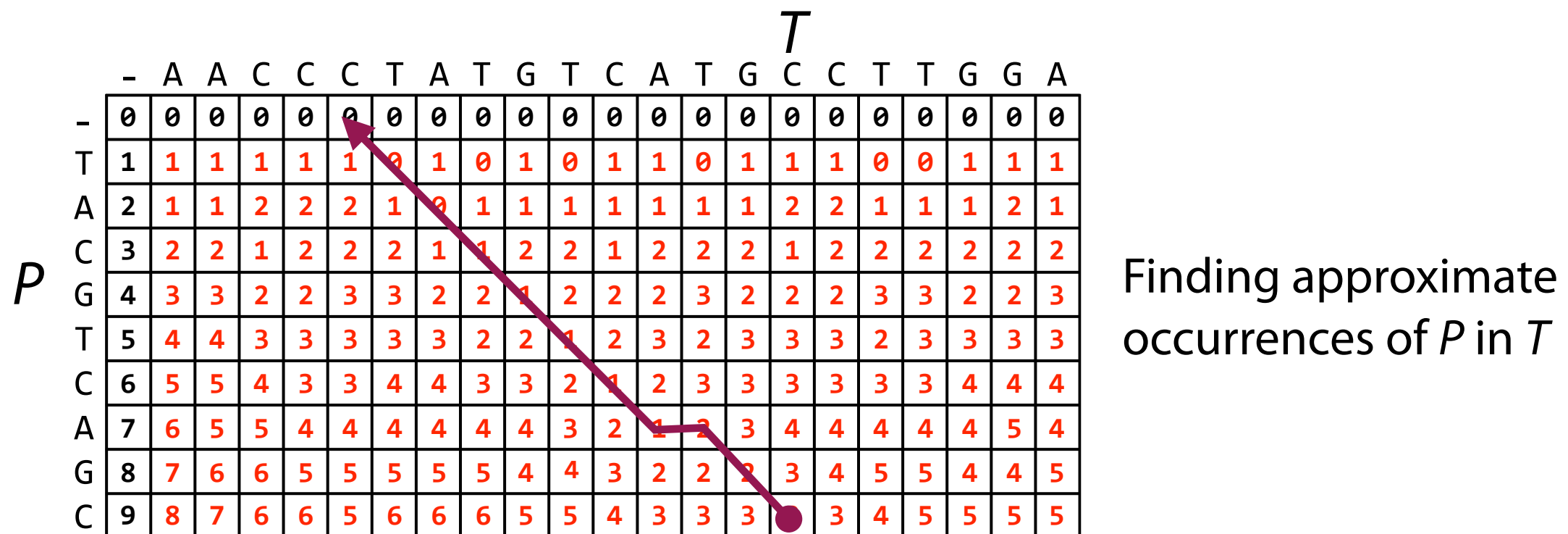
Please sign guestbook (www.langmead-lab.org/teaching-materials) to tell me briefly how you are using the slides. For original Keynote files, email me (ben.langmead@gmail.com).

Dynamic programming summary

A powerful set of tools:

- DP deals naturally with both mismatches and gaps
- DP scoring can take into account variation, sequencing error, etc

Along the way we saw an algorithm we might use for read alignment:



...but no faster than $O(mn)$ and m is big!!!

A de-motivating example

$$\left. \begin{array}{l} \mathbf{d} = 6 \times 10^9 \text{ reads} \\ \mathbf{n} = 100 \text{ nt} \\ \mathbf{m} = 3 \times 10^9 \text{ nt} \approx \text{human} \end{array} \right\} \approx 1 \text{ week-long run of}$$



Illumina HiSeq 2000

Say we have 1,000 processors, each clocked at 3 GHz, each capable of completing 8 dynamic programming cell updates per clock cycle
(We're being optimistic)

Total of $\mathbf{d} \times \mathbf{m} \times \mathbf{n} = 2 \times 10^{21}$ cell updates

Takes > 2 years

A de-motivating example

$$\left. \begin{array}{l} \mathbf{d} = 6 \times 10^9 \text{ reads} \\ \mathbf{n} = 100 \text{ nt} \\ \mathbf{m} = 3 \times 10^9 \text{ nt} \approx \text{human} \end{array} \right\} \approx 1 \text{ week-long run of}$$



Illumina HiSeq 2000

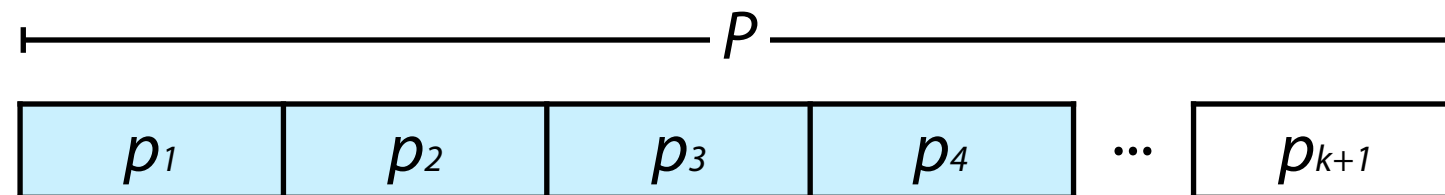
Problem: our dynamic programming approach is $O(dmn)$

We'll now consider two ideas for how to maintain the power of dynamic programming while diminishing effect of m

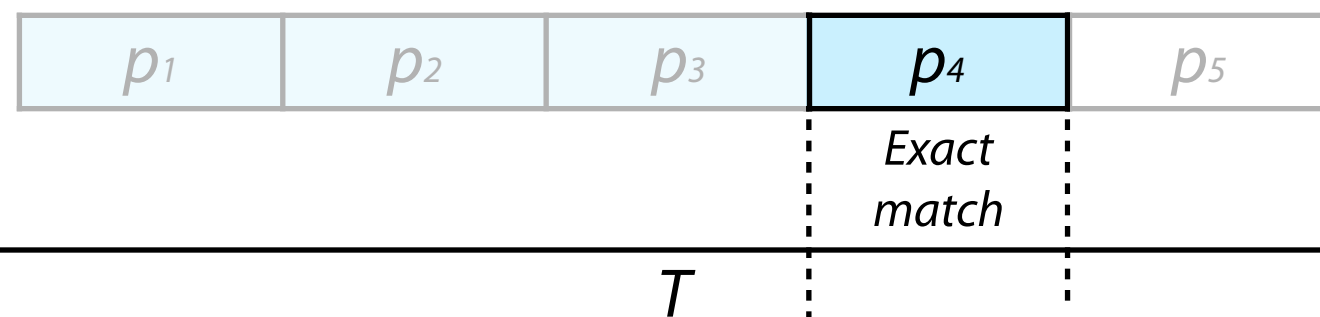
Index-assisted approximate matching

Idea 1: Use index for exact-matching subproblems, follow up with DP

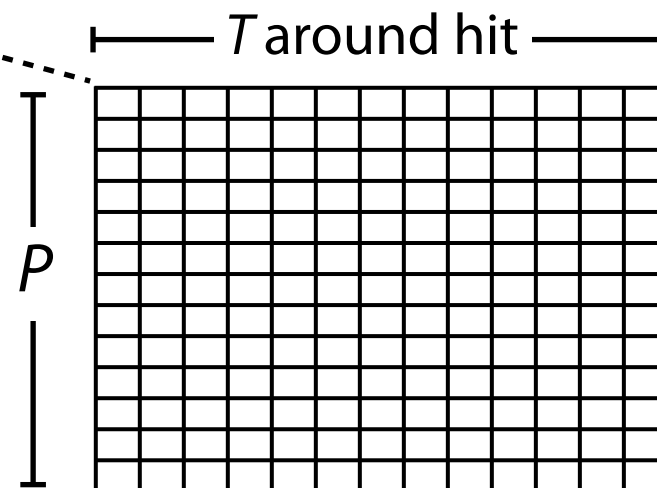
Partition P , like
for pigeonhole



Index finds exact
partition matches (hits)



Use DP in vicinity
of exact matches



Index-assisted approximate matching

Index-assisted function for finding occurrences of P in T with up to k edits:

```
def queryIndexEdit(p, t, k, index):  
    ''' Look for occurrences of p in t with up to k edits using an  
        index combined with dynamic-programming alignment. '''  
    l = index.ln  
    occurrences = []  
    seen = set() # for avoiding reporting same hit twice  
    for part, poff in partition(p, k+1):  
        for hit in index.occurrences(part): # query index w/ partition  
            # left edge of T to include in DP matrix  
            lf = max(0, hit - poff - k)  
            # right edge of T to include in DP matrix  
            rt = min(len(t), hit - poff + len(p) + k)  
            mn, off, xcript = kEditDp(p, t[lf:rt])  
            off += lf  
            if mn <= k and (mn, off) not in seen:  
                occurrences.append((mn, off, xcript))  
                seen.add((mn, off))  
    return occurrences
```

Partition P

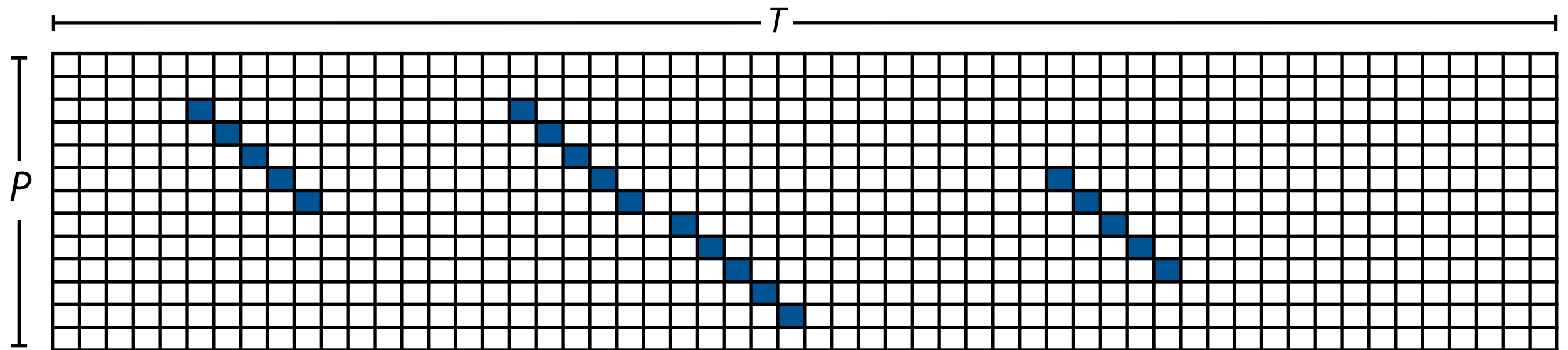
Query index

Dynamic programming

Python example: http://bit.ly/CG_kEdit_idx

Index-assisted approximate matching

Think in terms of the full P -to- T dynamic programming matrix



Index is identifying diagonal stretches of matches

These are likely to be part of a high-scoring alignment

Many stretches within a few diagonals of each other are even more likely to be part of a high-scoring alignment

Neighborhood search

Idea 2: Use index to find exact occurrences of strings in *P's neighborhood*

Neighborhood = set of strings within some Hamming / edit distance

The 1-edit neighborhood of cat, using DNA alphabet:

cat, aat, gat, tat, cct, cgt, ctt, caa, cac, cag, ca, ct, at, acat, ccat, gcat, tcat, ...

The diagram shows the 1-edit neighborhood of 'cat' with three categories indicated by brackets below the list of strings:

- All ways to add 1 mismatch:** This category includes the strings 'aat', 'gat', 'tat', 'cct', 'cgt', and 'ctt'.
- All ways to delete 1 char:** This category includes the strings 'ca', 'ct', and 'at'.
- All ways to insert 1 char:** This category includes the strings 'acat', 'ccat', 'gcat', and 'tcat'.

The original string 'cat' is not included in any of these categories, and an ellipsis '...' follows the last category.

The 2-mismatch neighborhood of cat:

cat, aat, gat, tat, cct, cgt, ctt, caa, cac, cag

The diagram shows the 2-mismatch neighborhood of 'cat' with a hierarchical structure indicated by arrows and brackets:

- The first level lists the strings: 'cat, aat, gat, tat, cct, cgt, ctt, caa, cac, cag'.
- A bracket below 'aat, gat, tat, cct, cgt, ctt' is labeled 'All ways to add 1 mismatch'.
- From 'aat', an arrow points to 'All ways to add 2nd mismatch to aat'.
- From 'gat', an arrow points to 'All ways to add 2nd mismatch to gat'.
- An arrow from the bottom points to '...'.

Neighborhood search

Idea 2: Use index to find occurrences of strings in P 's "neighborhood"

Is the neighborhood huge? Can we bound it?

If $|P| = n$, and $|\Sigma| = a$, how many strings are within Hamming distance 1?

$$1 + \underbrace{n(a-1)}_{a-1 \text{ ways to replace each of } P\text{'s } n \text{ chars}}$$

$\nearrow$
 P itself

How many strings are within edit distance 1?

$$1 + n(a-1) + \underbrace{n + (n+1)a}_{\substack{n+1 \text{ positions where we can} \\ \text{insert any of the } a \text{ characters}^*}}$$

$\nearrow$
Delete each char in P *

In both cases, $O(an)$ strings in the neighborhood

* Some insertions are equivalent. E.g. there are two equivalent insertions of 'a' into 'cat'. Likewise deletions ('caat').

Neighborhood search

How about within Hamming or edit distance 2?

$O(an)$ strings within Hamming or edit distance 1, each with $O(an)$ neighbors within distance 1, so $O(a^2n^2)$

Within distance k ?

$$O(a^kn^k)$$

How much work to query suffix tree with all strings within distance k ?

$O(n + \# \text{ occurrences})$ for each of the $O(a^kn^k)$ strings, so roughly $O(a^kn^{k+1})$

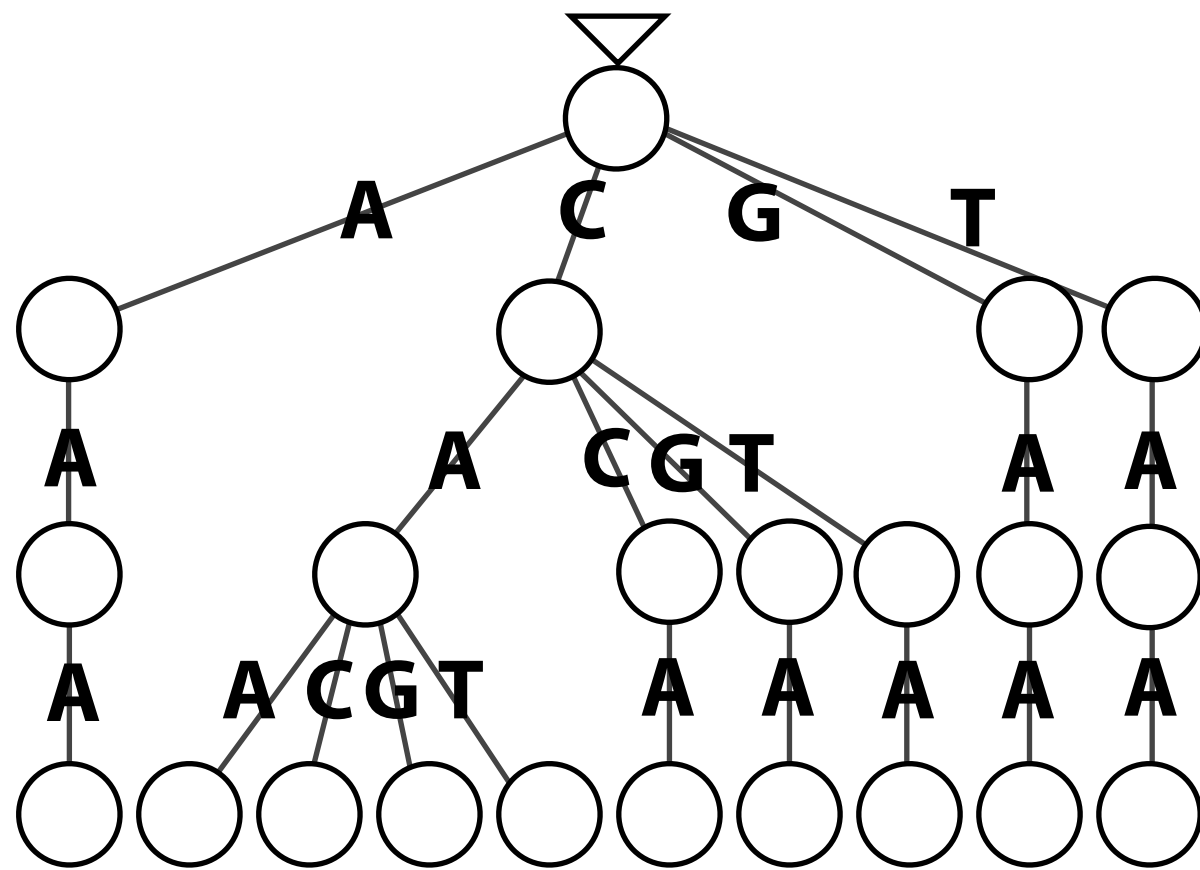
Compare to $O(a^kn^{k+1})$ to $O(mn)$ for full dynamic programming

Good: no m Bad: exponential in k

Neighborhood search

Organize neighborhood of P into a trie

Neighbors of $P = CAA$, within hamming distance 1:



I # leaves = # neighbors =
 $1 + n(a - 1) = 1 + 3(4 - 1) = 10$

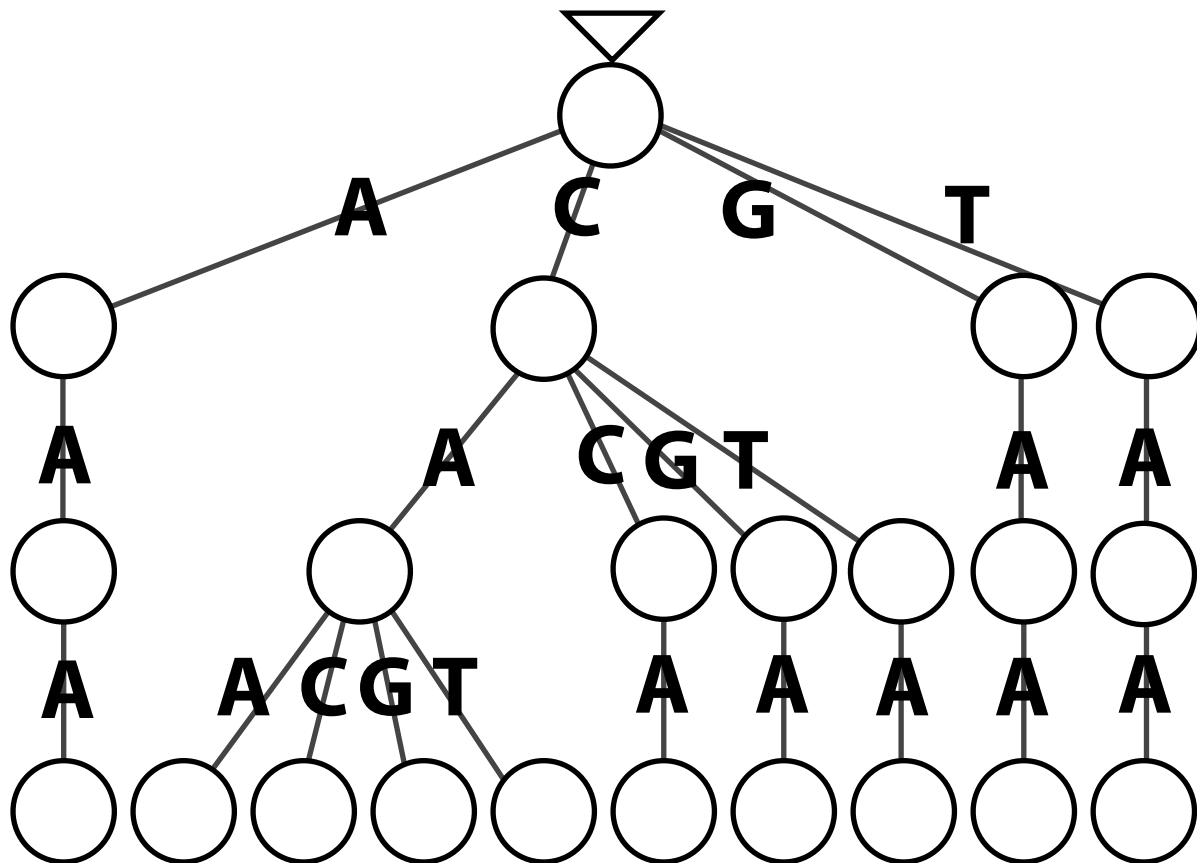
Neighborhood search

Navigating and/or building neighborhood trie is simple with recursion

Assume Hamming distance for now: Move left-to-right across P and start with “budget” of k mismatches

At each step, for each alphabet character c :

- If c matches current character in P , recursively build subtree starting at next position of P with same budget
- If c *mismatches* current character in P *and* budget > 0 , recursively build subtree starting at next position of P with 1 subtracted from budget. Otherwise if budget $= 0$, move on.



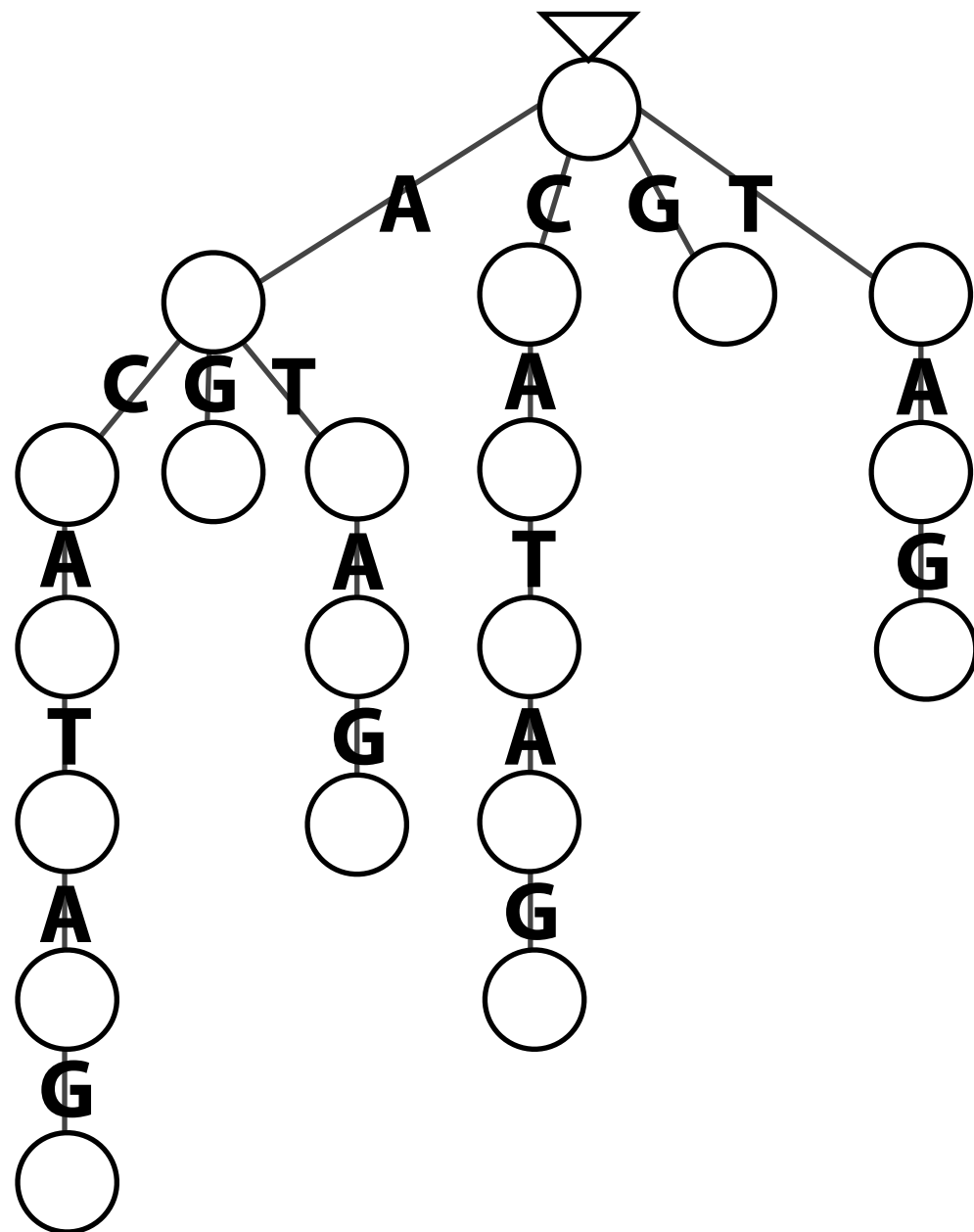
Neighborhood search

```
>>> stringNeighbors("cat", "acgt", edits=1, gaps=False)
['aat', 'gat', 'tat', 'cct', 'cgt', 'ctt', 'caa', 'cac', 'cag', 'cat']
>>> stringNeighbors("cat", "acgt", edits=1, gaps=True)
['acat', 'ccat', 'gcat', 'tcat', 'at', 'aat', 'gat', 'tat', 'caat',
'ccat', 'cgat', 'ctat', 'ct', 'cct', 'cgt', 'ctt', 'caat', 'cact',
'cagt', 'catt', 'ca', 'caa', 'cac', 'cag', 'cata', 'catc', 'catg',
'catt', 'cat']
```

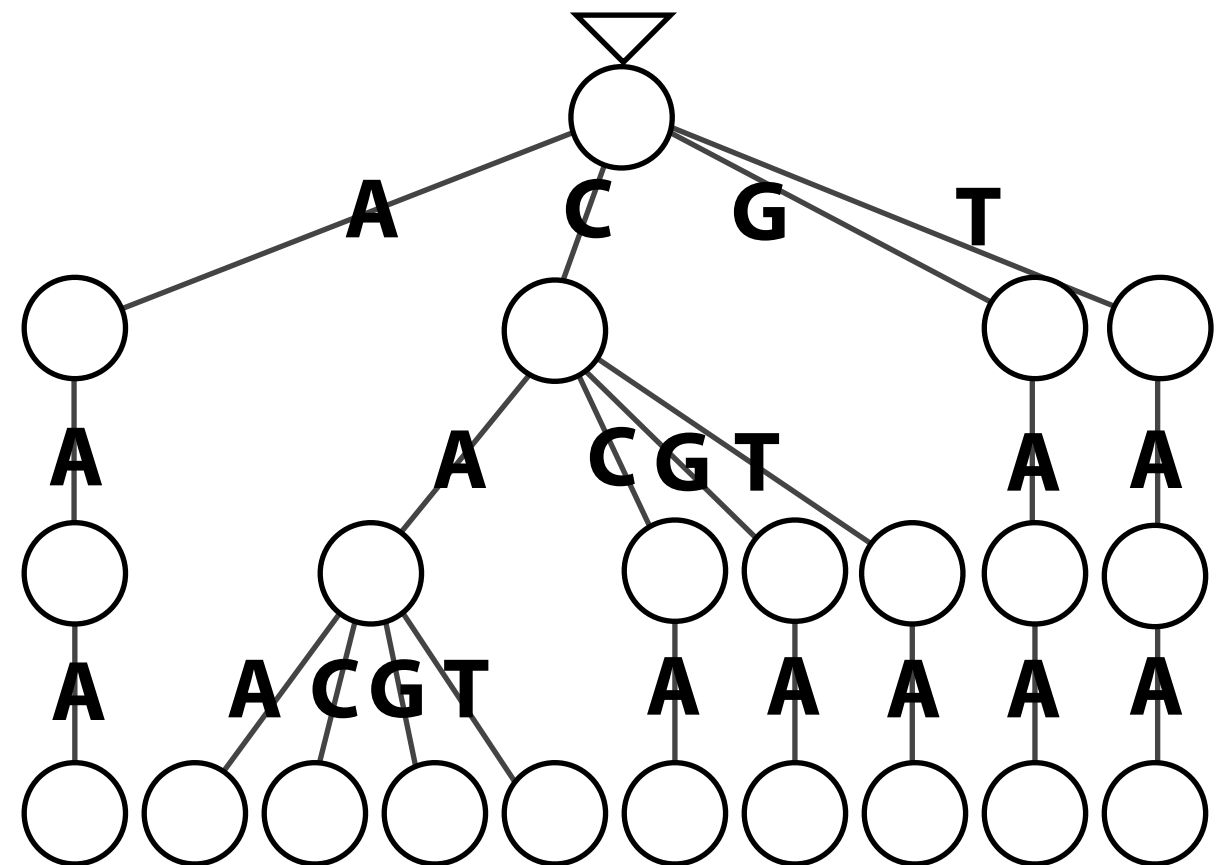

Co-traversal

Say index is a suffix trie. Imagine querying it with each neighbor.

Suffix trie of $T = \text{ACATAG}$



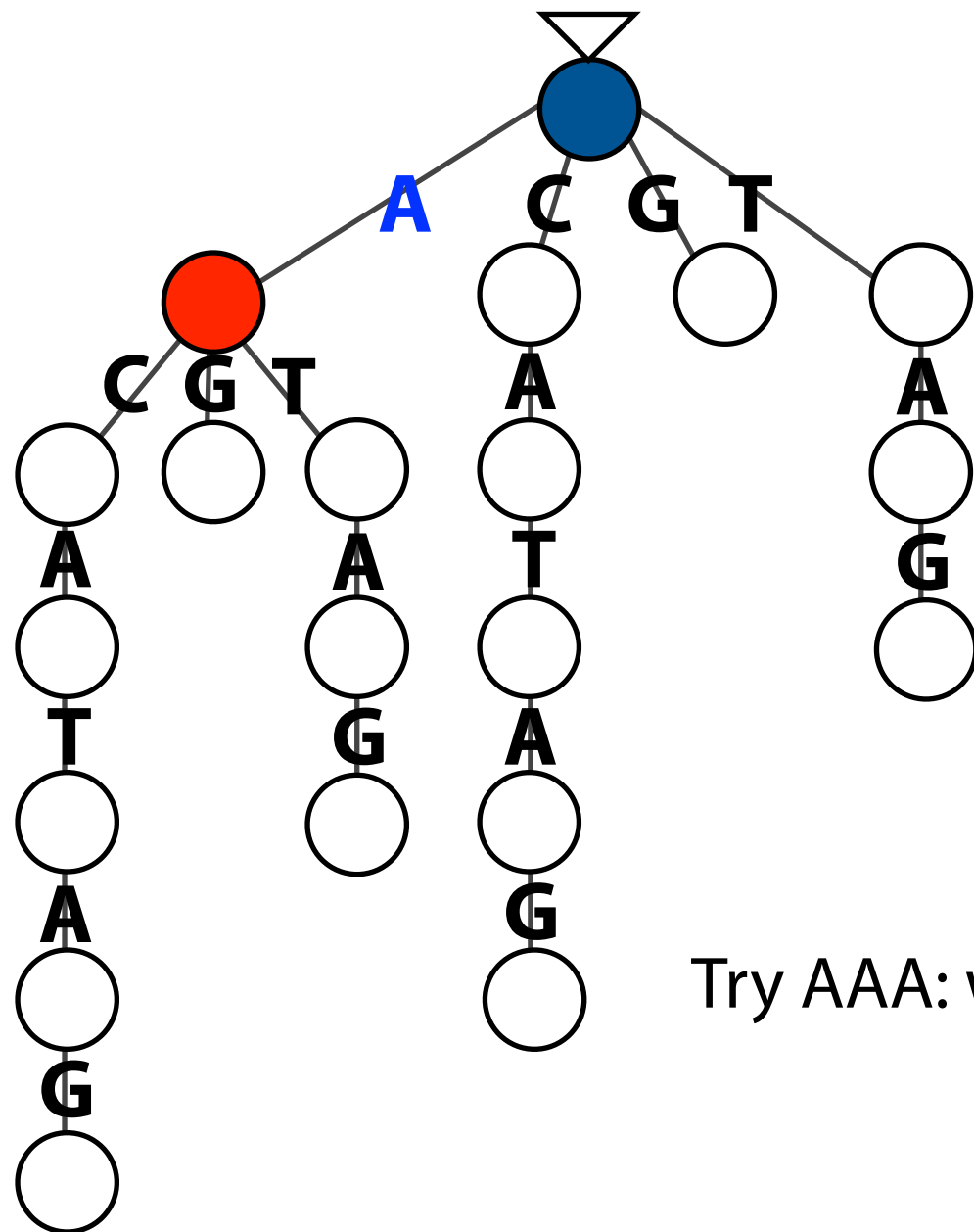
Trie for neighborhood within
1 mismatch of $P = \text{CAA}$



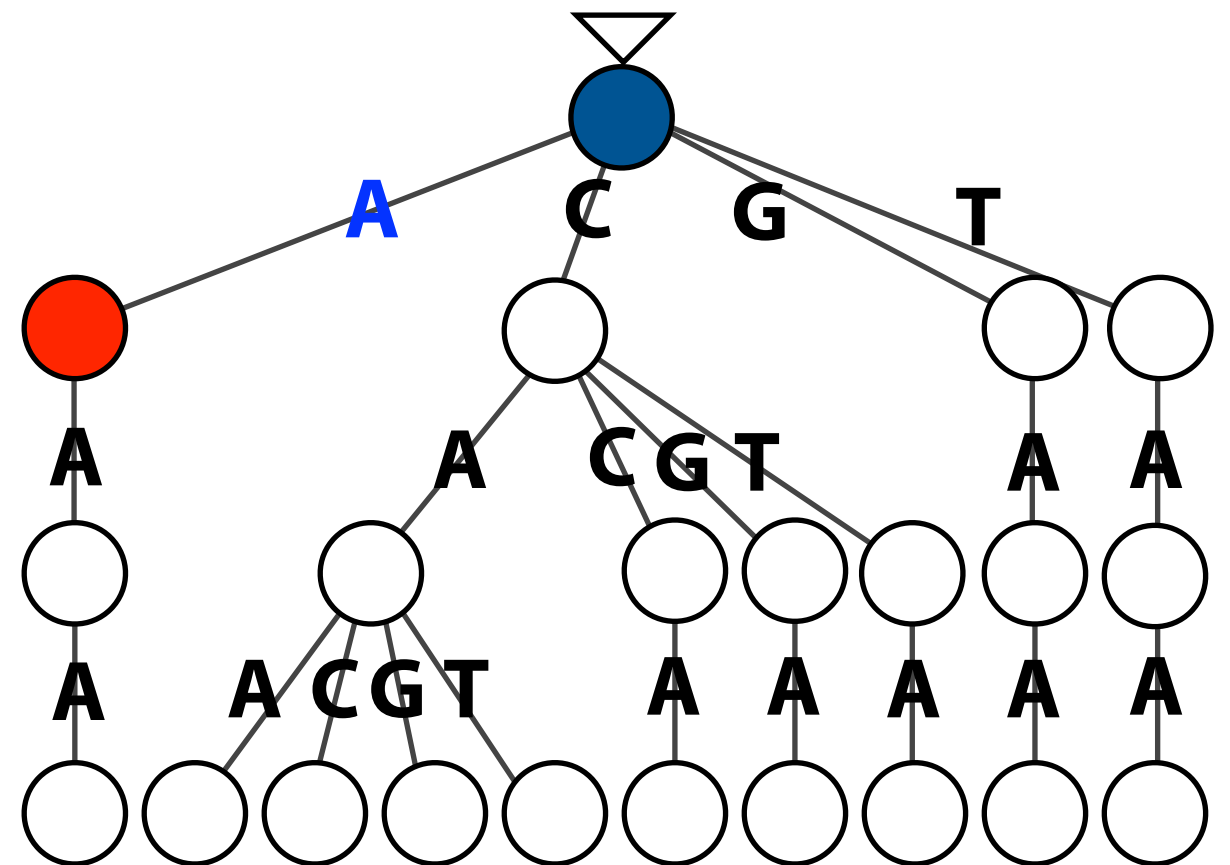
Co-traversal

Say index is a suffix trie. Imagine querying it with each neighbor.

Suffix trie of $T = \text{ACATAG}$



Trie for neighborhood within 1 mismatch of $P = \text{CAA}$

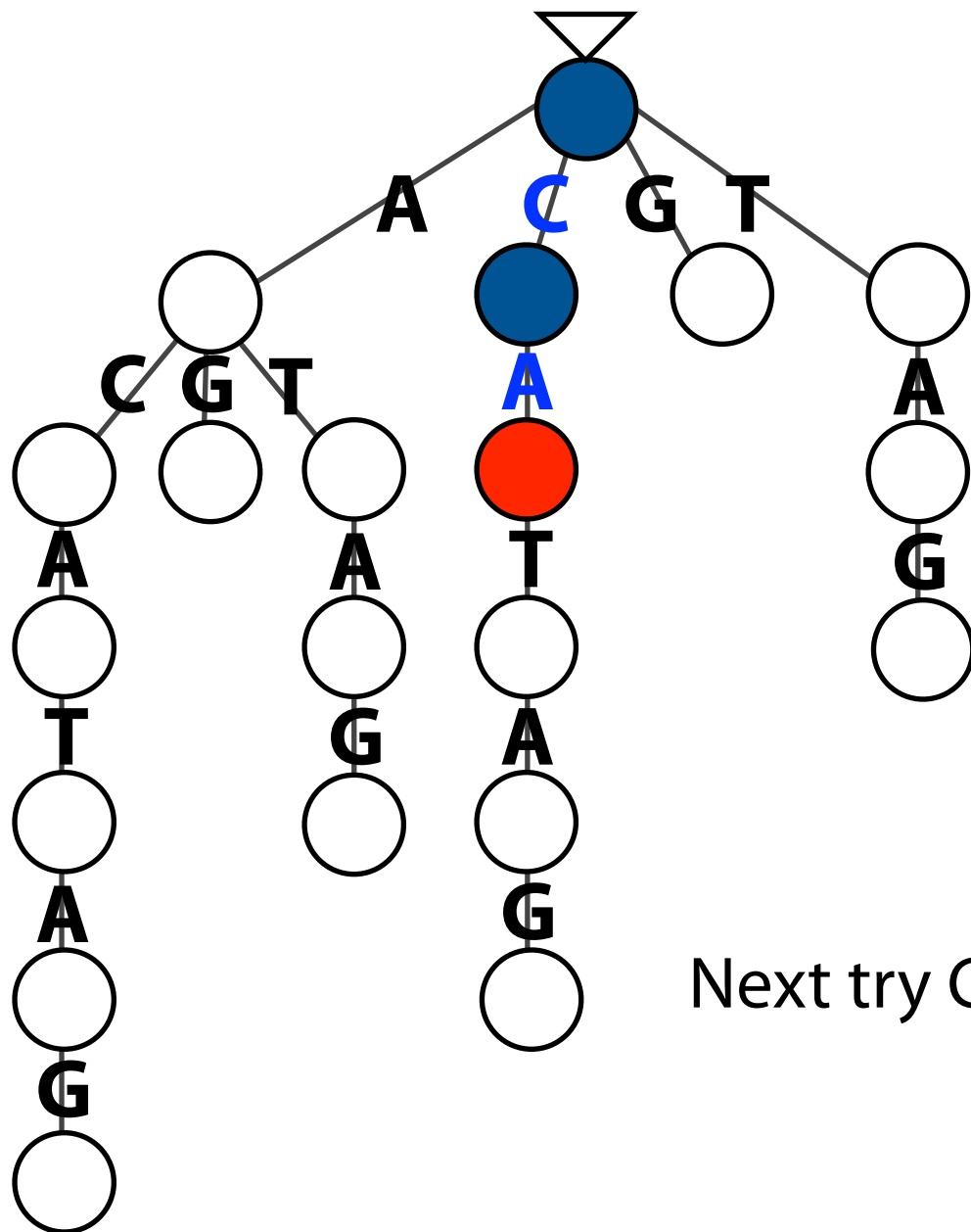


Try AAA: we fall off suffix trie after A, before AA

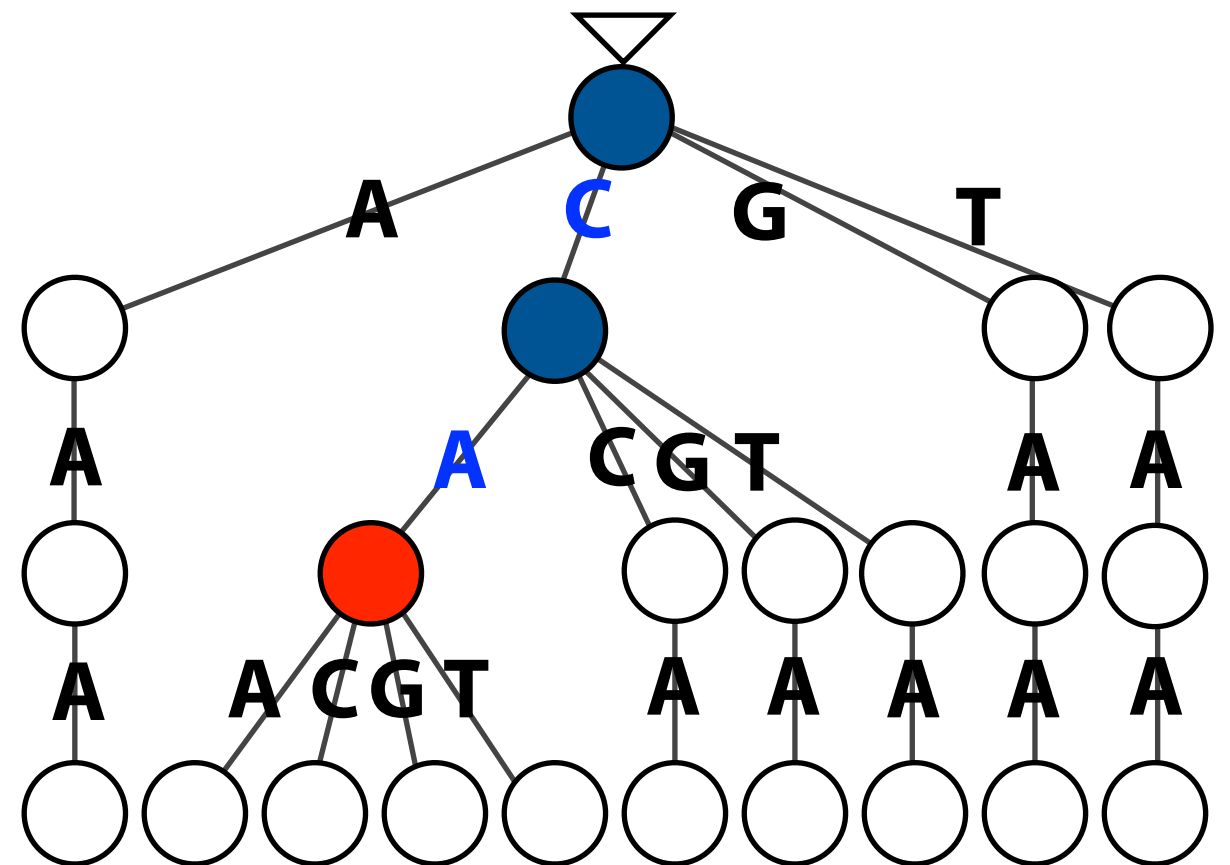
Co-traversal

Say index is a suffix trie. Imagine querying it with each neighbor.

Suffix trie of $T = \text{ACATAG}$



Trie for neighborhood within 1 mismatch of $P = \text{CAA}$

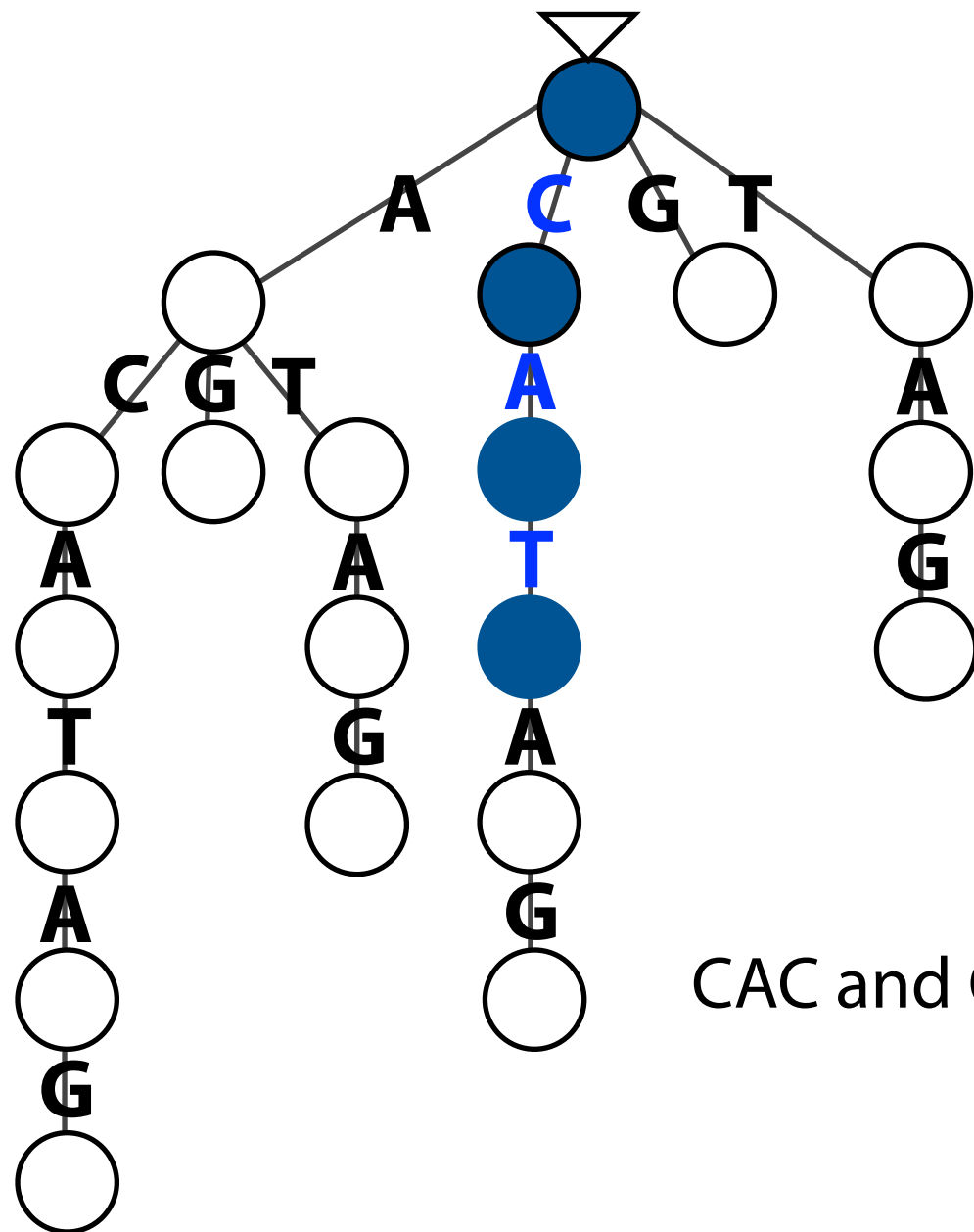


Next try CAA: we fall off suffix trie after CA, before CAA

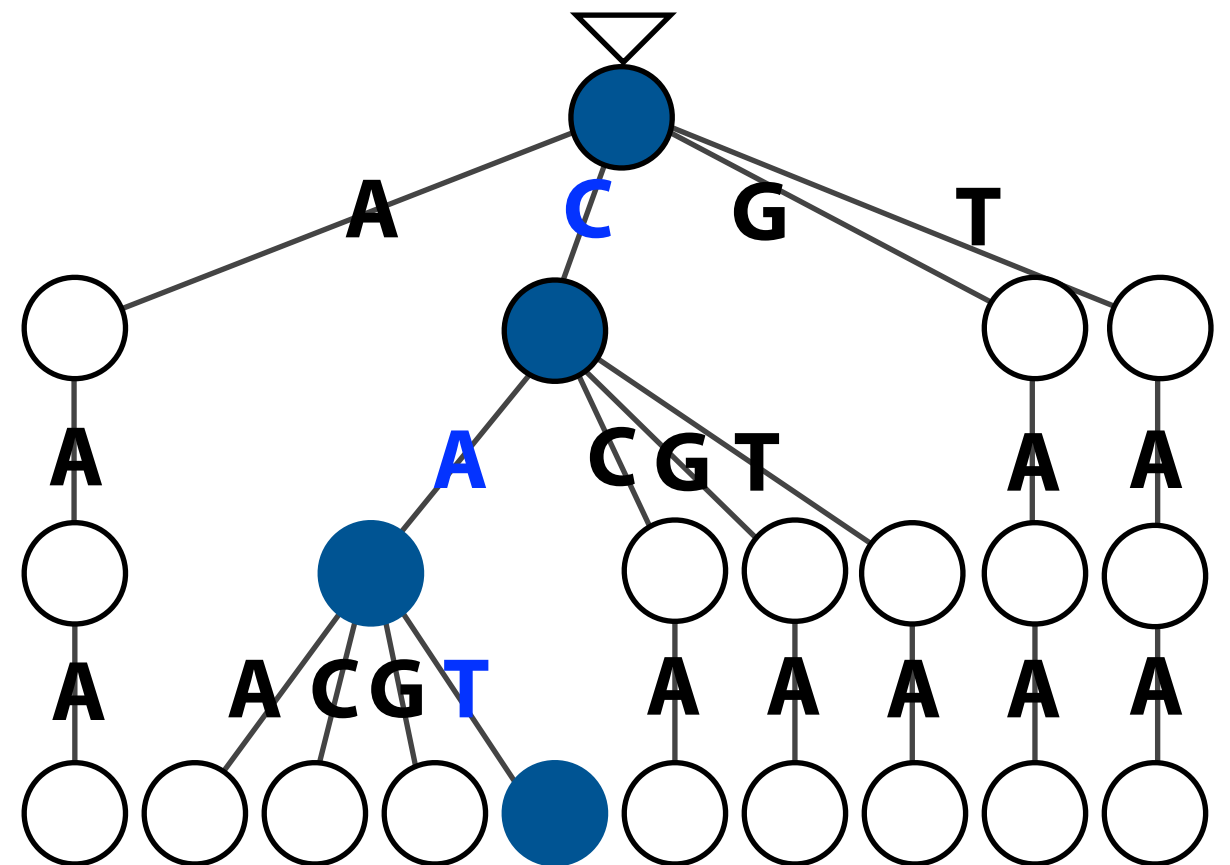
Co-traversal

Say index is a suffix trie. Imagine querying it with each neighbor.

Suffix trie of $T = \text{ACATAG}$



Trie for neighborhood within 1 mismatch of $P = \text{CAA}$

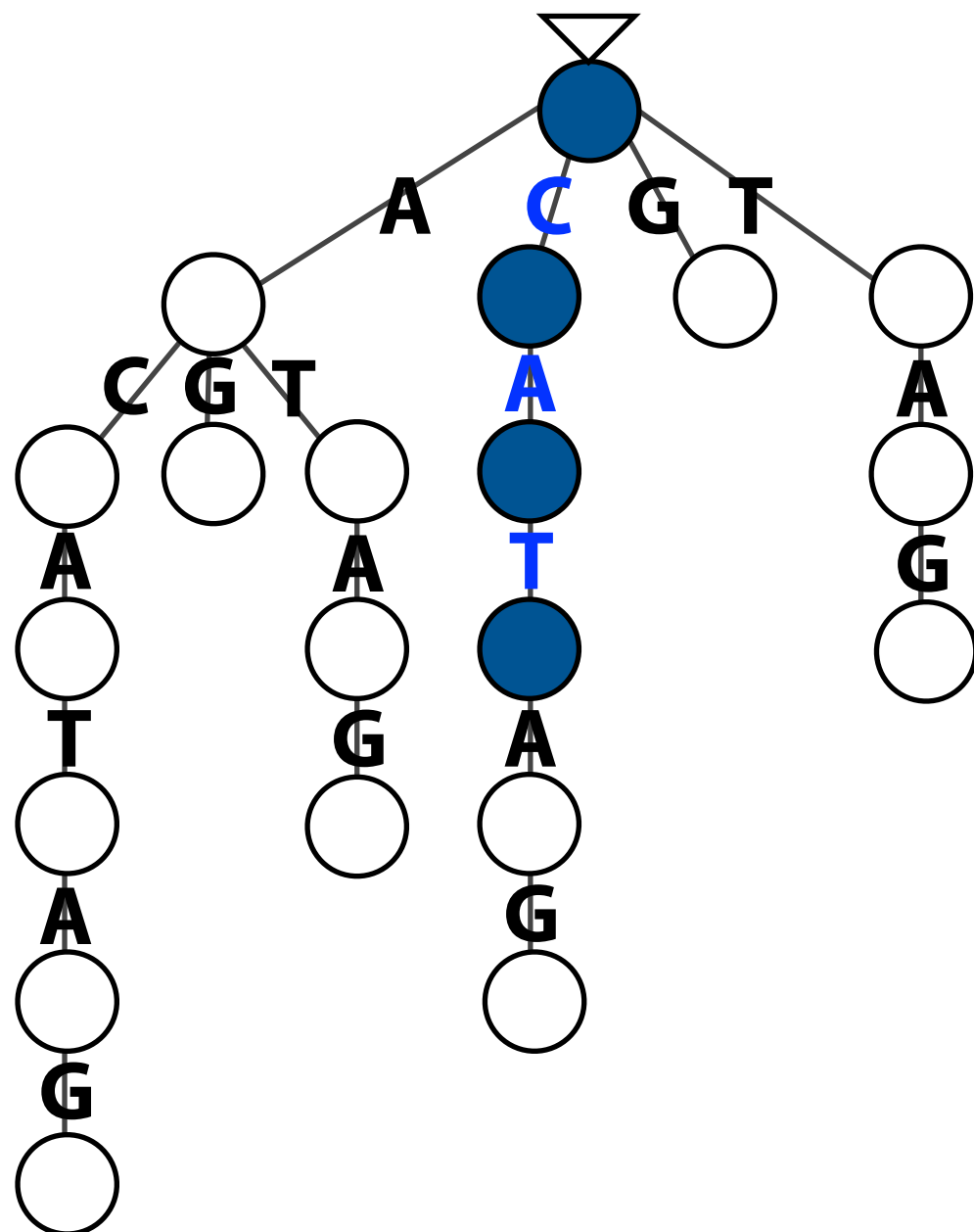


CAC and CAG also fail. Next try CAT: success.

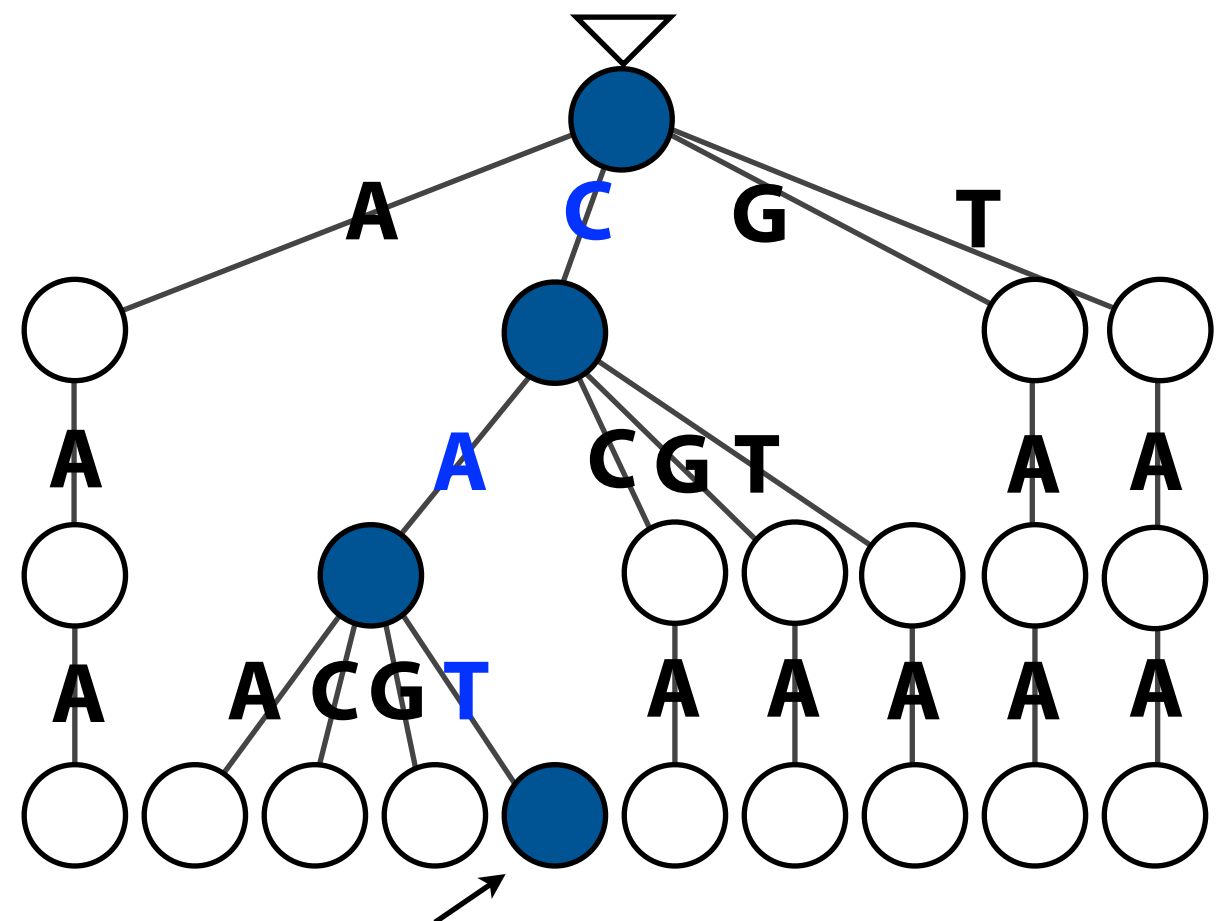
Co-traversal

Say index is a suffix trie. Imagine querying it with each neighbor.

Suffix trie of $T = \text{ACATAG}$



Trie for neighborhood within 1 mismatch of $P = \text{CAA}$



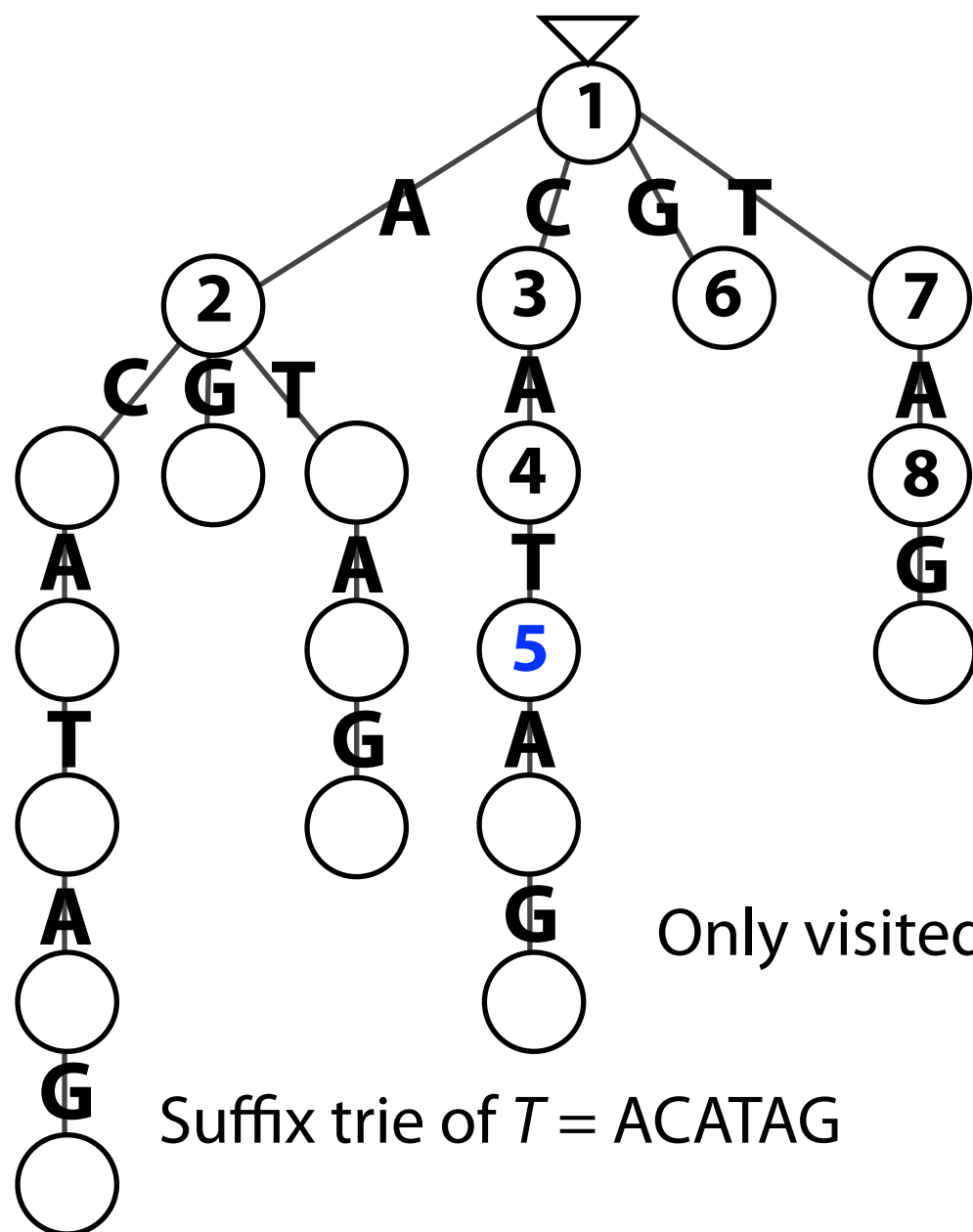
Common path ending in a neighbor leaf corresponds to an alignment of a neighbor of P to a substring of T

T : **A** **C** **A** **T** **A** **G**
/ /
 P : **C** **A** **A**

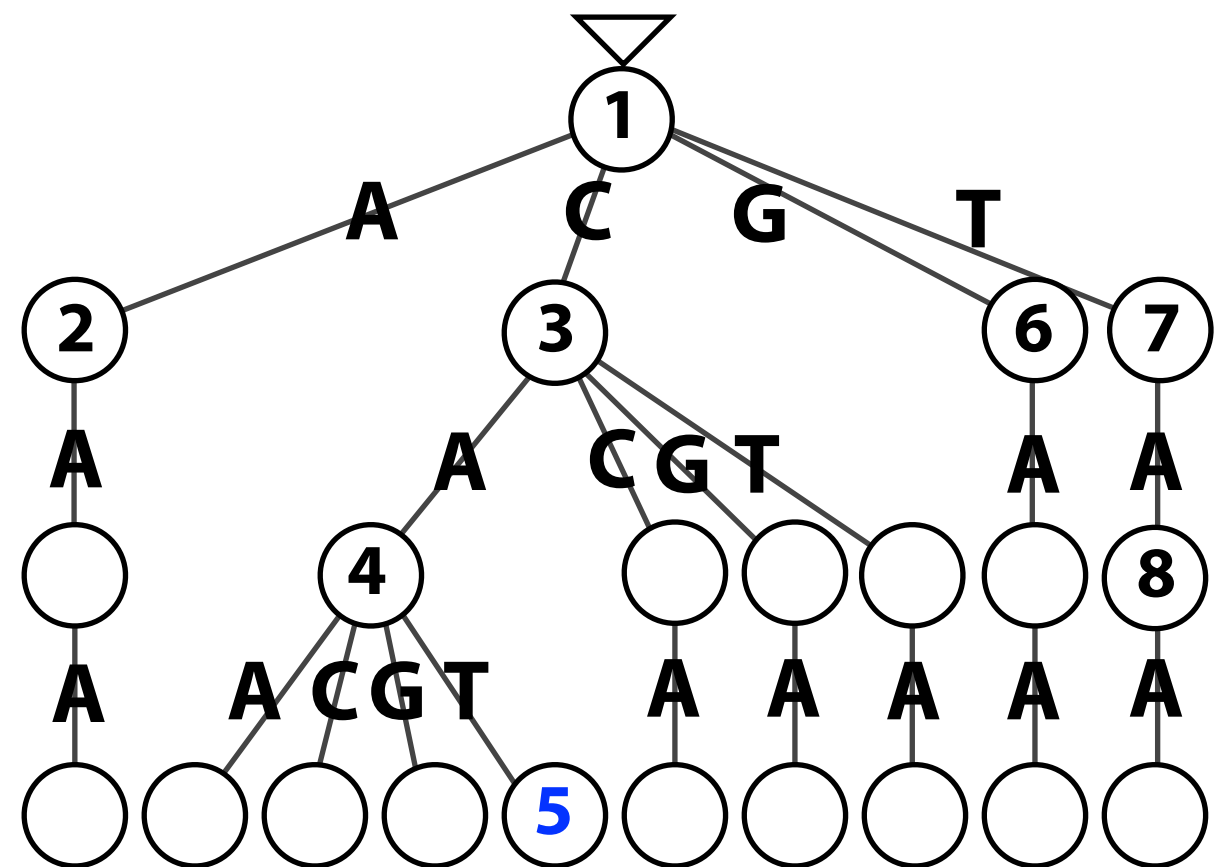
Co-traversal

We can find all such alignments with *co-traversal*: explore all paths that are present in both trees and end in a neighbor leaf

Lexicographical depth-first co-traversal visits node pairs in this order:



Suffix trie of $T = ACATAG$

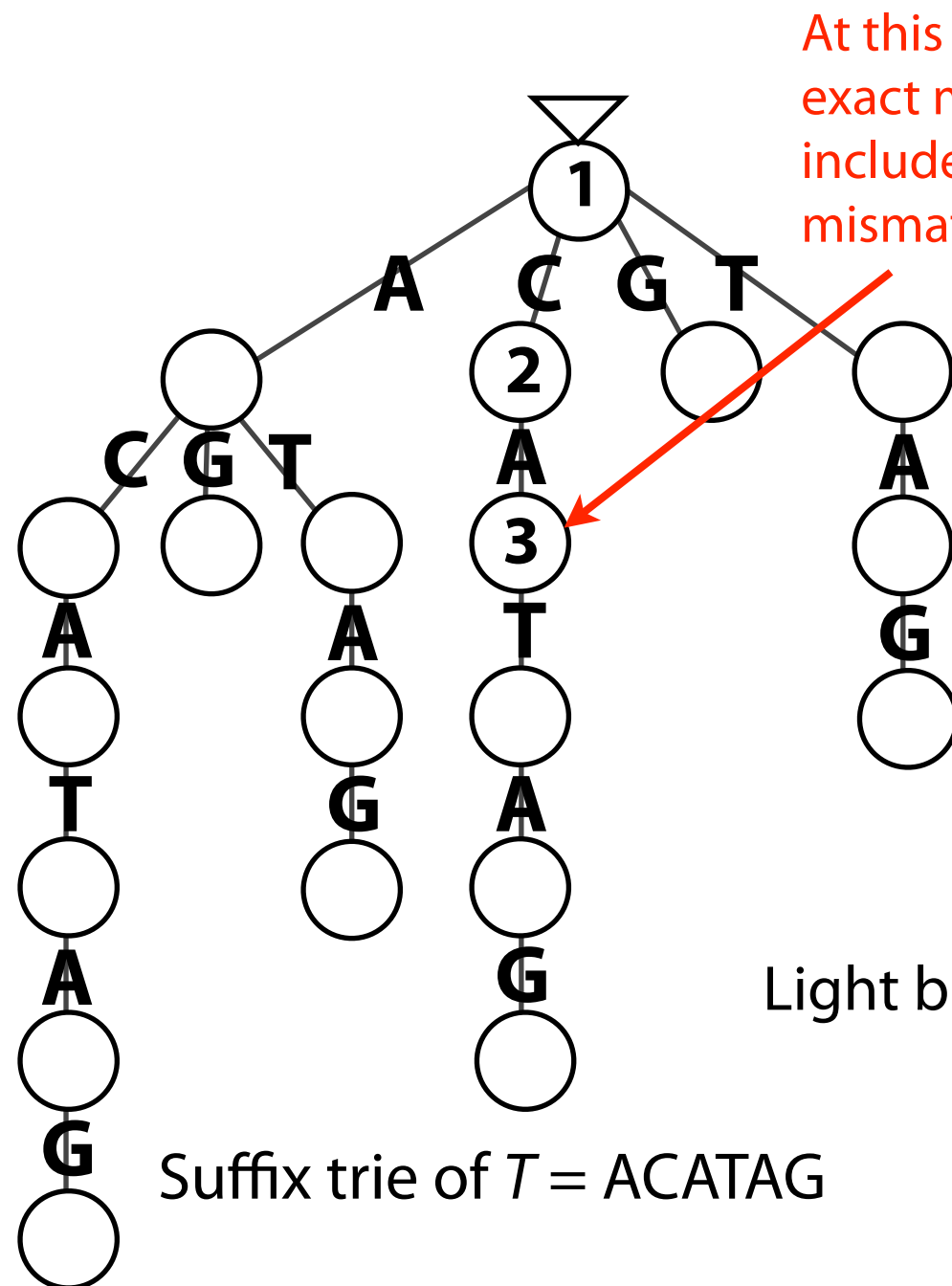


Only visited 8 nodes in these 20- and 22-node tries

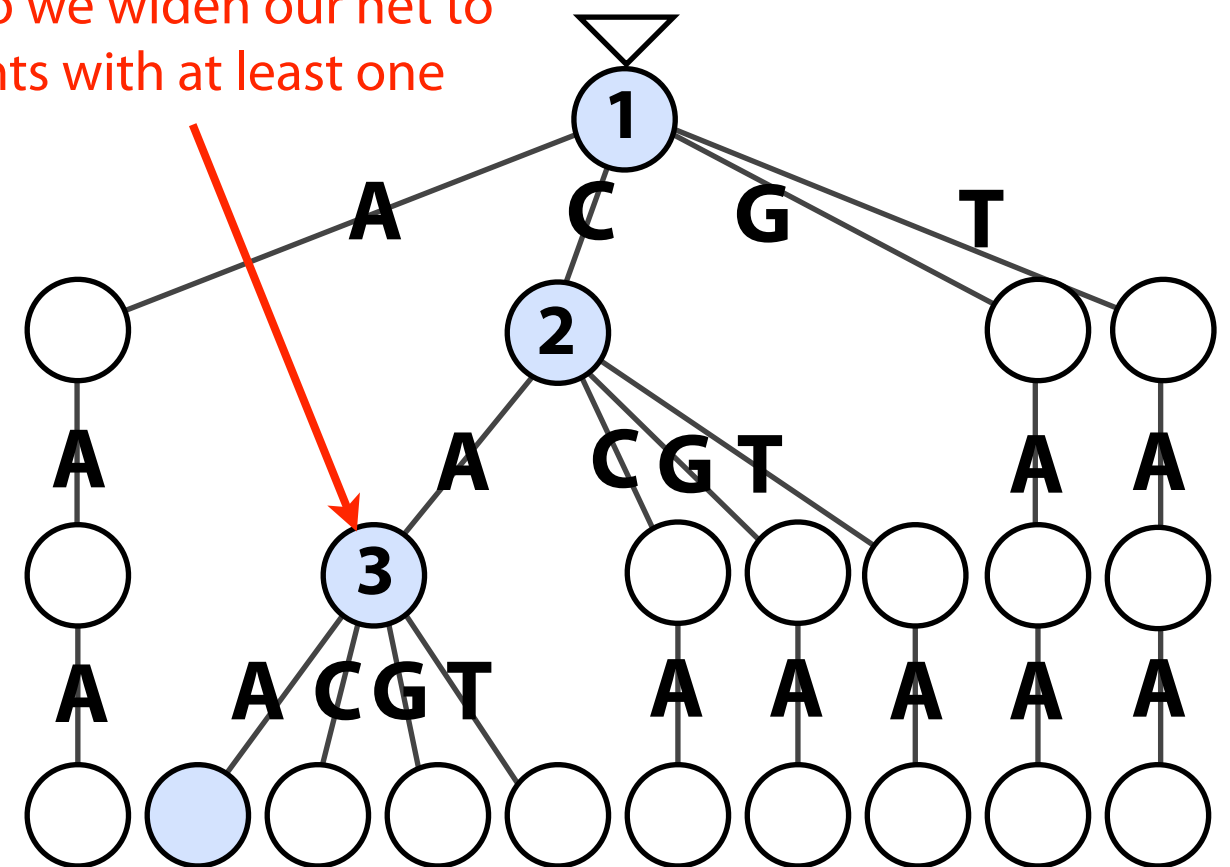
Trie for neighborhood within
1 mismatch of $P = CAA$

Co-traversal

We can also conduct *best-first* search, visiting paths with fewer edits before paths with more edits:



At this point we know there are no exact matches, so we widen our net to include alignments with at least one mismatch

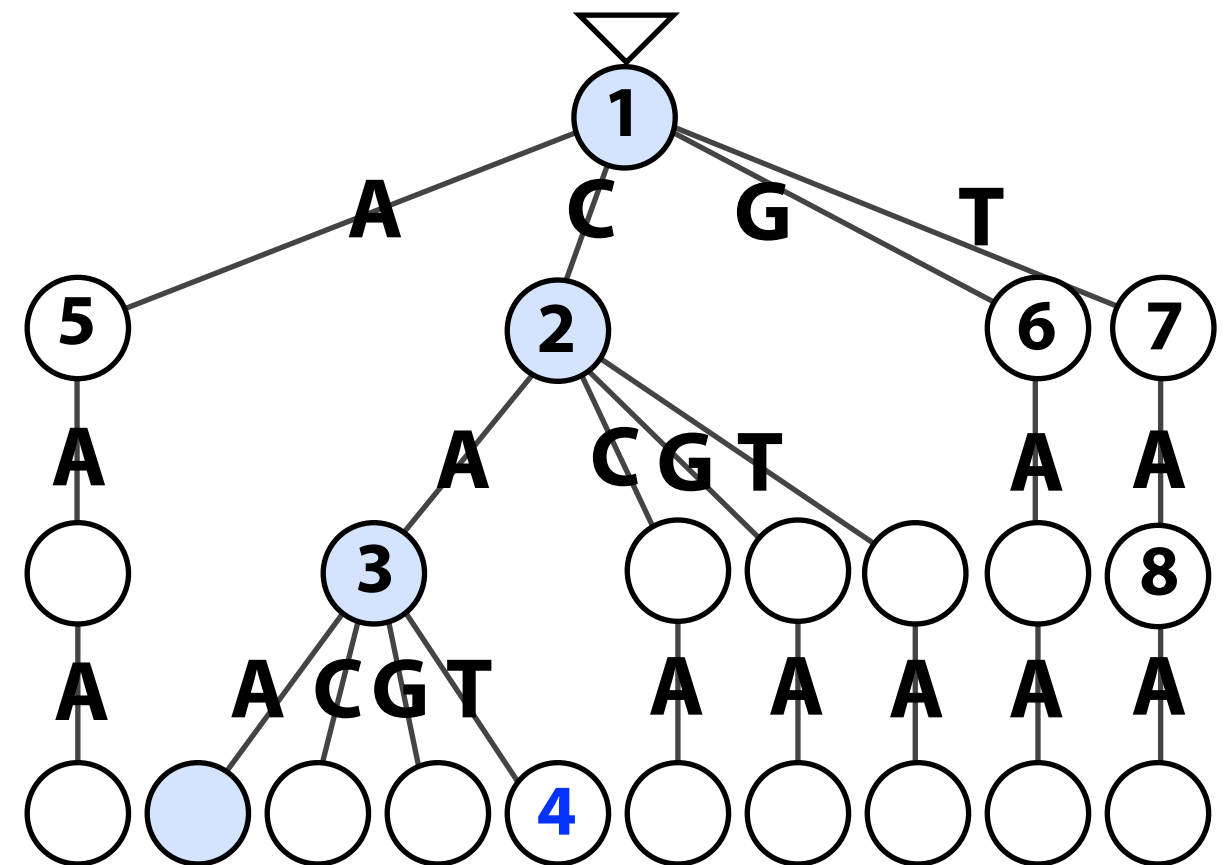
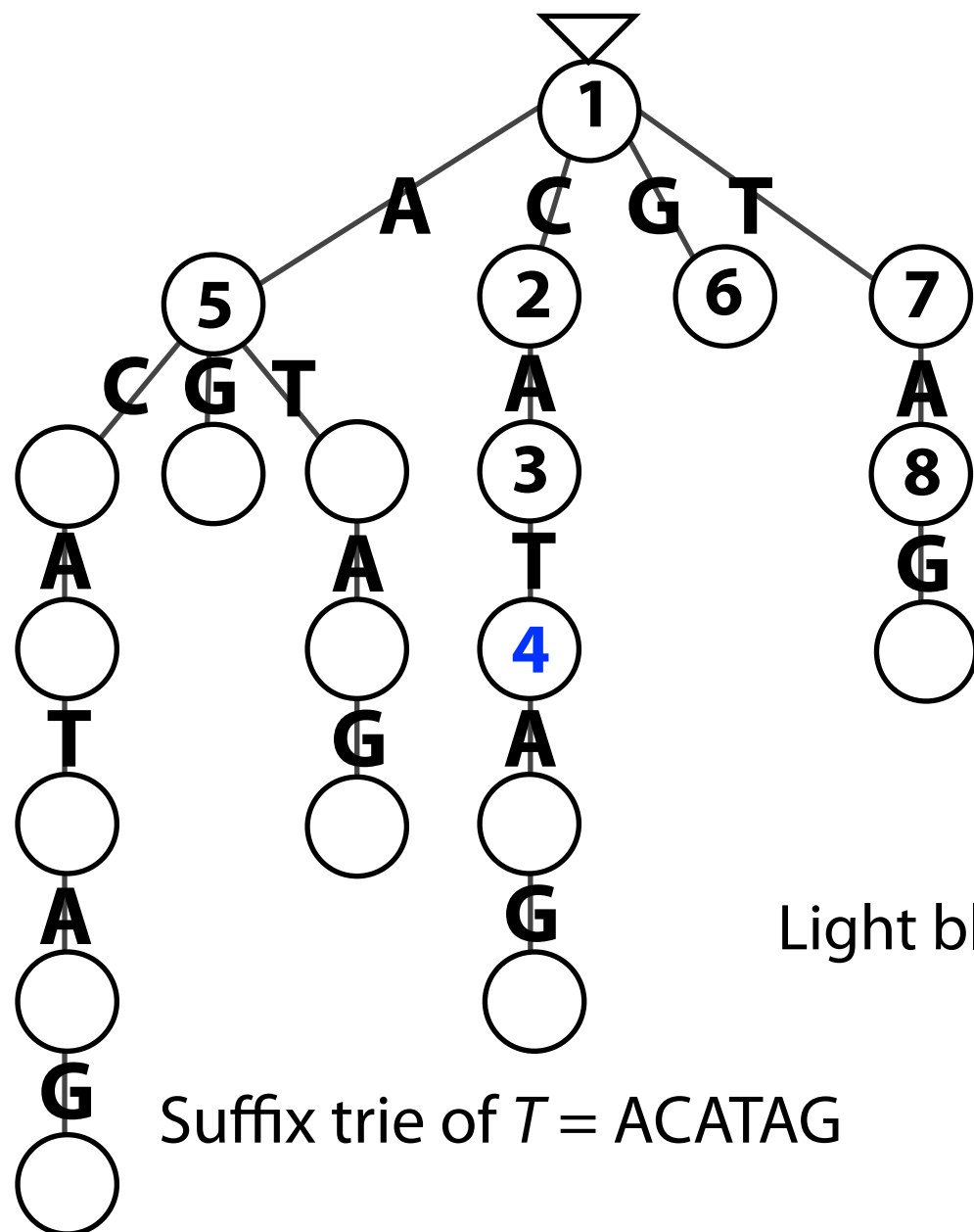


Light blue nodes match P exactly. Others have 1 mismatch.

Trie for neighborhood within
1 mismatch of $P = CAA$

Co-traversal

We can also conduct *best-first* search, visiting paths with fewer edits before paths with more edits:

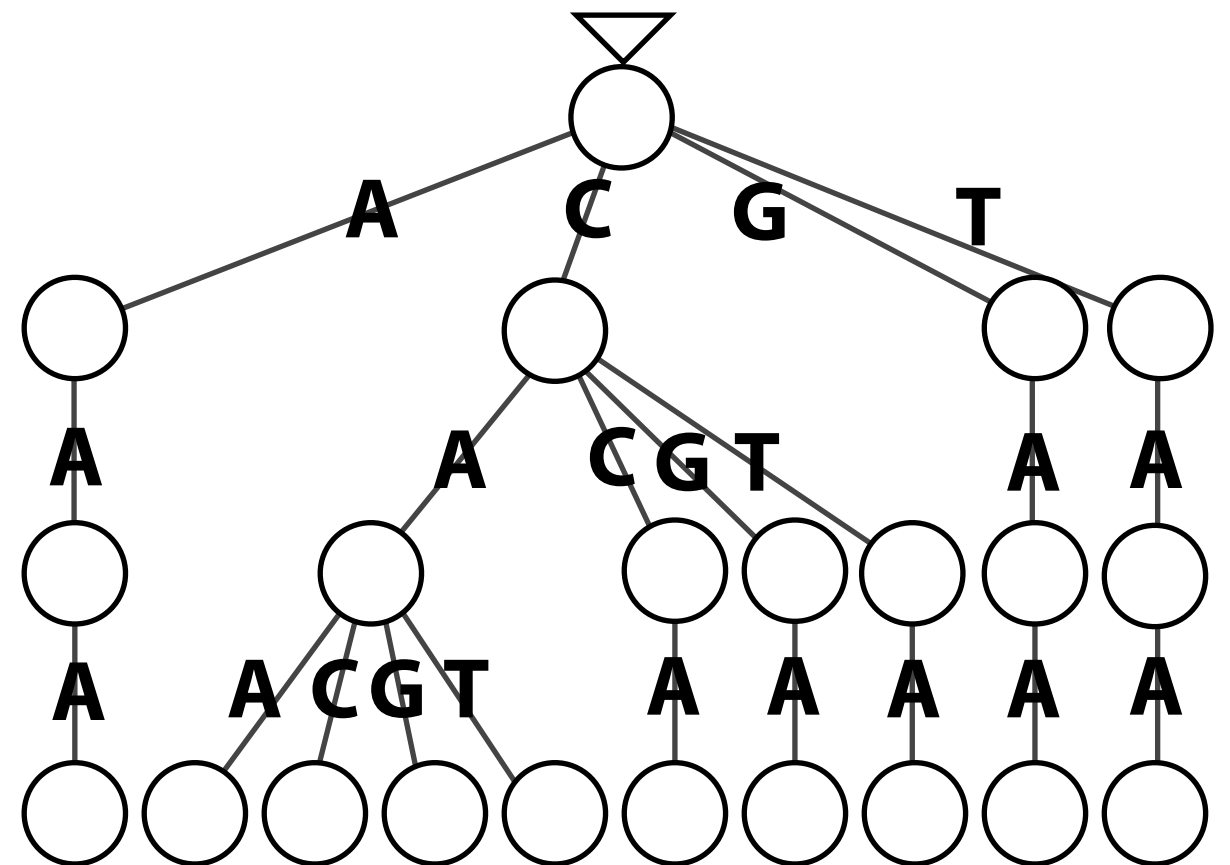
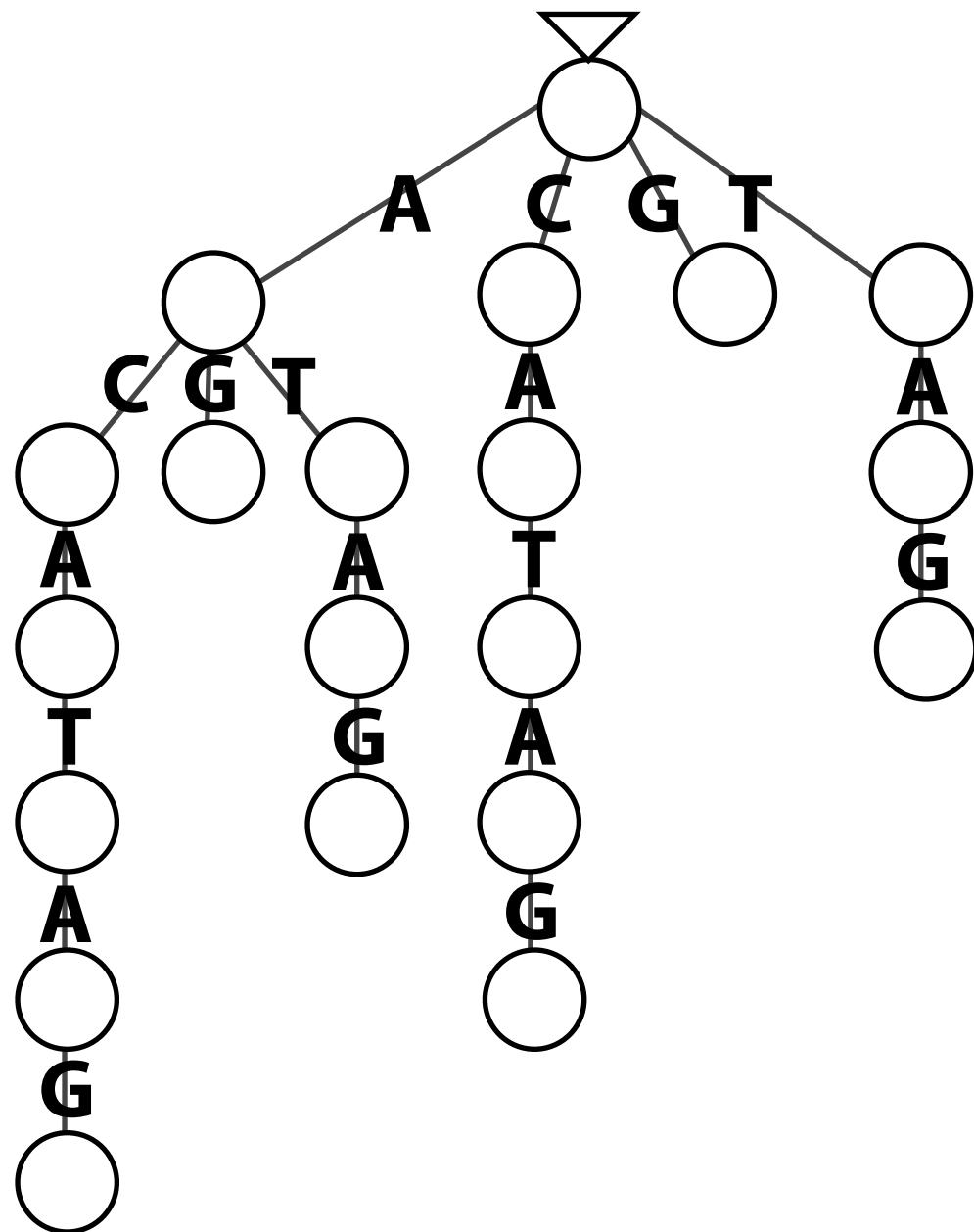


Light blue nodes match P exactly. Others have 1 mismatch.

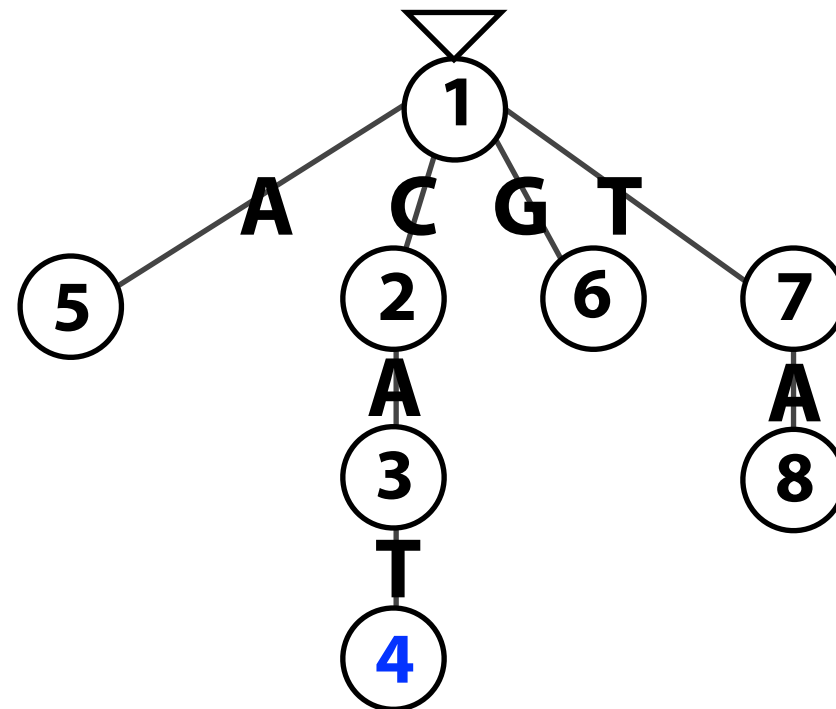
Trie for neighborhood within
1 mismatch of $P = CAA$

Co-traversal: pruning

We can think of the tree we're exploring as being the *intersection* of these two trees...



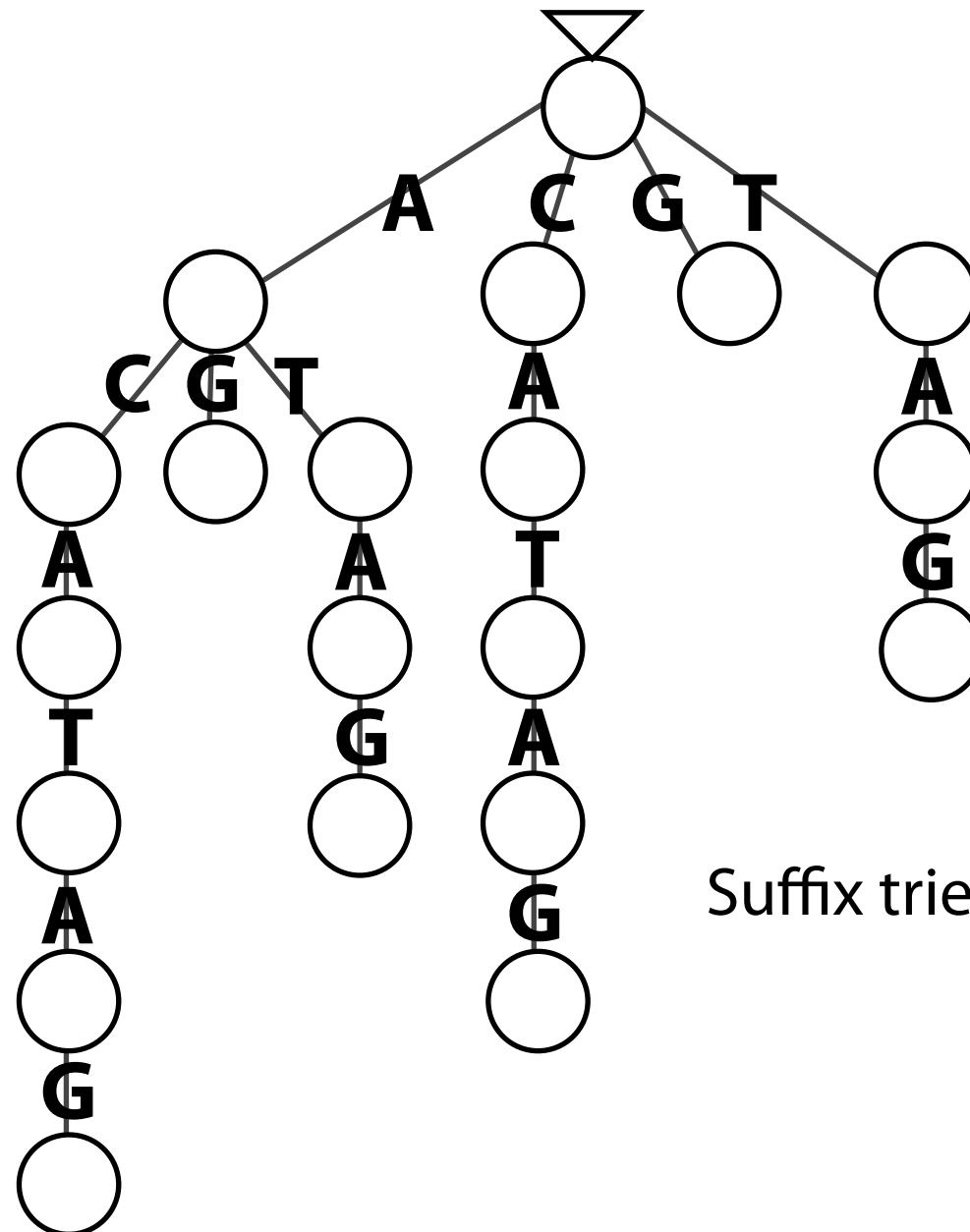
Co-traversal: pruning



Suffix trie of $T = ACATAG$ $\cap$ Trie for neighborhood within
1 mismatch of $P = CAA$

Co-traversal: indexing

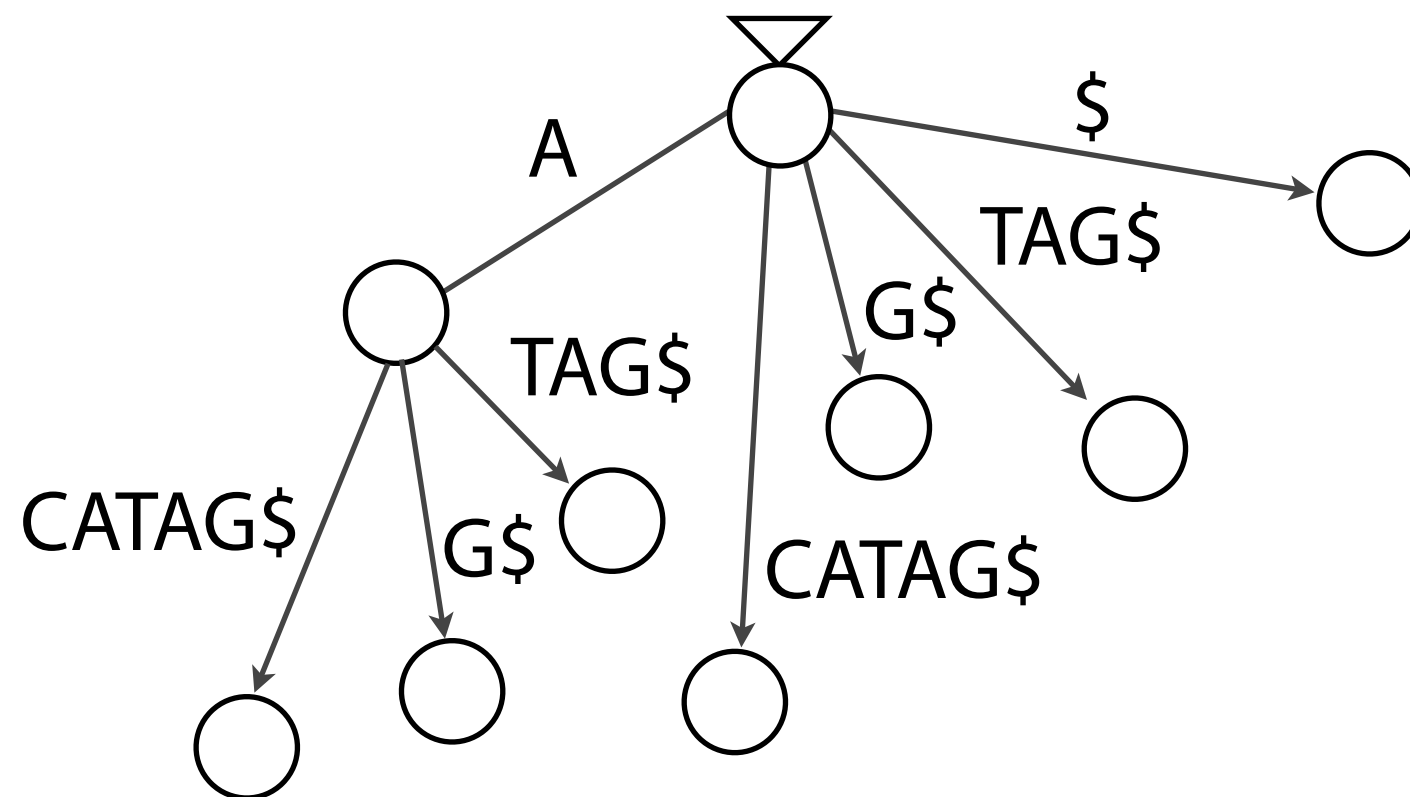
Co-traversal uses the shape of the suffix trie, but we don't want to actually build it. It's $O(m^2)$ space. What's an alternative?



Suffix trie of $T = \text{ACATAG}$

Co-traversal: indexing

Alternative 1: Replace suffix trie with suffix tree



Suffix tree of $T = ACATAG\$$

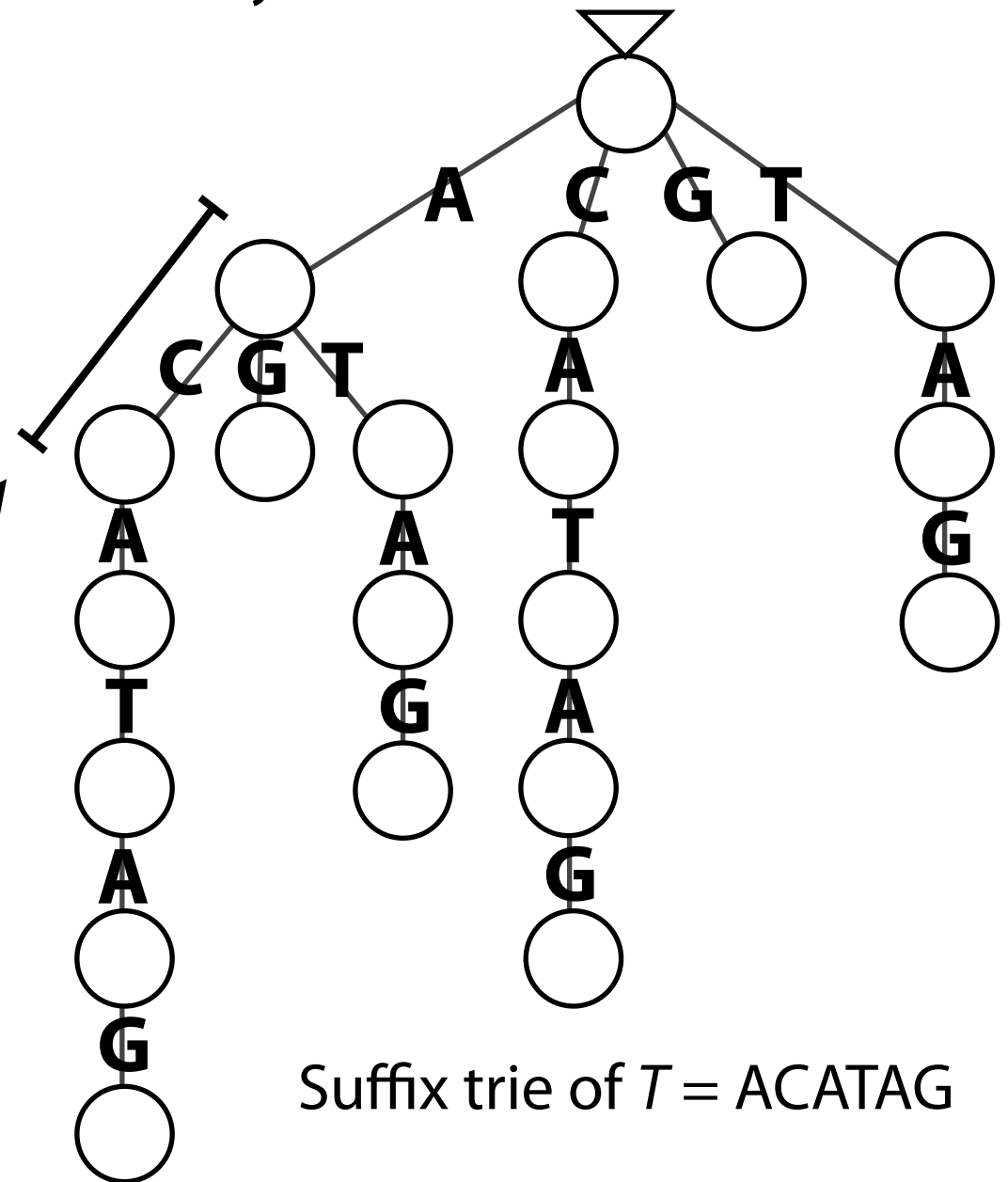
Co-traversal: indexing

Alternative 2: Replace suffix trie with suffix array

6	\$
0	A C A T A G \$
4	A G \$
2	A T A G \$
1	C A T A G \$
5	G \$
3	T A G \$

If we know range of SA elements with A as a prefix, additional binary searching gives range with AC as a prefix

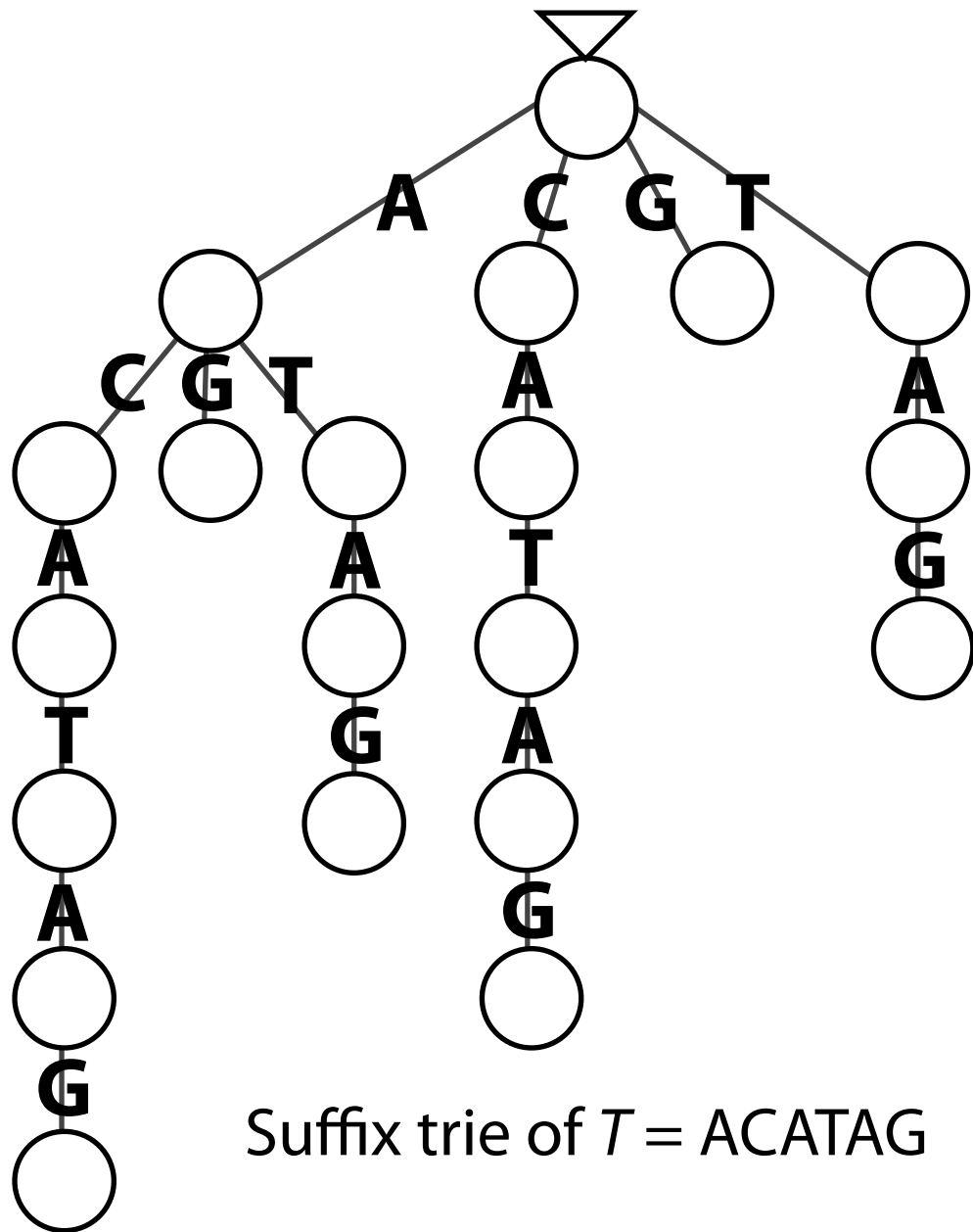
Even if we don't *build* suffix trie/tree, suffix array allows us to *traverse* it



Suffix trie of $T = ACATAG$

Co-traversal: indexing

Alternative 3: Replace suffix trie with FM Index



Similar argument as suffix array, using LF Mapping instead of binary search

To traverse suffix trie, we need to build FM Index of $T' = \text{reverse}(T)$

Why?

Typical FM Index matches successively longer *suffixes* of P . If we want to match successively longer *prefixes*, we have to reverse T before building FM Index.